TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH

BÁO CÁO CUỐI KỲ

Môn học: Phân tích dữ liệu lớn

Phân tích dữ liệu cảm biến và dự đoán tuổi thọ hữu ích còn lại của động cơ

NASA C-MAPSS FD001

Đề tài 8: Phân tích dữ liệu IoT và bảo trì dự đoán

Học viên thực hiện:	Phan Nguyễn Mai Phương
Mã số học viên:	25002711
Ngành:	Khoa học máy tính

Bản đang thực hiện – cập nhật ngày 22/09/2026

TP. Hồ Chí Minh, tháng 9 năm 2026

Tóm tắt trạng thái báo cáo

Báo cáo nghiên cứu bài toán dự đoán Remaining Useful Life (RUL, tuổi thọ hữu ích còn lại) theo đơn vị cycle trên NASA C-MAPSS FD001. Đến ngày 22/09/2026, repository đã có dữ liệu nguồn, data contract (đặc tả dữ liệu và các điều kiện hợp lệ), gói kiểm tra dữ liệu, nhãn RUL train và các artifact EDA (Exploratory Data Analysis, phân tích khám phá dữ liệu). Báo cáo validation ghi nhận 20.631 quan sát train và 13.096 quan sát test; mỗi tập có 100 động cơ.

Bản hiện tại trình bày bài toán, cơ sở lý thuyết và nghiên cứu liên quan, cách kiểm tra chất lượng, thiết kế phương pháp và các quan sát EDA có nguồn đối chiếu. M2.3a so sánh trung bình 30 cycles đầu với 30 cycles cuối của từng động cơ train; notebook đã xuất bảng cặp, bảng suy luận, metadata và biên bản validation. Pipeline Silver/Gold, mô hình RUL, ARIMA và dashboard chưa hoàn thành. Vì chưa có dự đoán và metric mô hình, báo cáo chưa đưa ra kết luận về độ chính xác hoặc hiệu quả bảo trì.

Tiến độ được ghi theo công việc và bằng chứng, không quy đổi thành phần trăm hoàn thành khi chưa có cơ sở ước lượng. Các ngày ghi nhận là mốc có tài liệu hoặc artifact hỗ trợ, không phải lịch sử giờ công được tái dựng. Nội dung về phương pháp chưa triển khai được đánh dấu rõ để tiếp tục cập nhật sau mỗi mốc.

Mục lục

Tóm tắt trạng thái báo cáo **i**

Bảng thuật ngữ và chữ viết tắt **iv**

1 Tiến độ thực hiện và kế hoạch tiếp theo **1**
1.1 Cách ghi nhận tiến độ . . . 1
1.2 Các mốc tiếp theo và điều kiện hoàn thành . . . 3

2 Bài toán, mục tiêu và phạm vi **5**
2.1 Bài toán nghiên cứu . . . 5
2.2 Câu hỏi và giới hạn nghiên cứu . . . 5
2.3 Liên hệ với yêu cầu môn học . . . 5

3 Dữ liệu và kiểm soát chất lượng **6**
3.1 Nguồn dữ liệu và lý do lựa chọn . . . 6
3.2 Từ điển dữ liệu và ý nghĩa cột . . . 6
3.3 Hồ sơ cột trên tập train . . . 7
3.4 Các kiểm tra đã triển khai và lý do . . . 9
3.5 Tính toàn vẹn và giới hạn của validation . . . 9

4 Cơ sở lý thuyết và nghiên cứu liên quan **10**
4.1 Prognostics, RUL và phạm vi so sánh . . . 10
4.2 Biểu diễn chuỗi bằng trailing window . . . 10
4.3 Baseline và Ridge regression . . . 11
4.4 Histogram gradient boosting . . . 11
4.5 Đơn vị độc lập và đánh giá theo engine . . . 11
4.6 Paired analysis, bootstrap và nhiều kiểm định . . . 12
4.7 Health Index từ PCA và phân tích ARIMA . . . 13
4.8 Spark, lưu trữ và giới hạn của benchmark . . . 13

5 Thiết kế phương pháp và lý do lựa chọn **14**
5.1 Luồng xử lý và trạng thái triển khai . . . 14
5.2 EDA theo đơn vị động cơ . . . 15
5.3 Đặc trưng từ window M3.2 . . . 15
5.4 Mô hình và đánh giá: phần dự kiến . . . 16
5.5 Thống kê hiện có và ARIMA dự kiến . . . 16

6 Protocol thống kê suy luận mốc 2 **18**
6.1 Mục tiêu, phạm vi và trạng thái quyết định . . . 18
6.2 Định nghĩa cặp đầu/cuối và ví dụ . . . 18
6.3 Đại lượng cần báo cáo và điều kiện diễn giải . . . 19
6.4 Cấu hình M2.0 đã chốt trước triển khai . . . 20
6.5 Đầu ra và điều kiện nghiệm thu . . . 21

7 Kết quả hiện có và diễn giải **22**
7.1 Kết quả validation . . . 22
7.2 ETL Silver/Gold và benchmark . . . 22
7.3 Rolling features M3.2 . . . 22
7.4 Tuổi thọ động cơ trên train . . . 23
7.5 Sensor hằng và correlation theo engine . . . 24
7.6 Suy luận đầu/cuối vòng đời trên train . . . 25
7.7 Phạm vi bằng chứng EDA và thống kê . . . 29

8 Khó khăn, bài học và giới hạn hiện tại **30**

9 Khai báo sử dụng AI/LLM **30**

Tài liệu tham khảo **31**

A Bảng đối chiếu nội dung và bằng chứng **33**

B Thông tin cần bổ sung trước khi nộp **35**

Bảng thuật ngữ và chữ viết tắt

Bảng dưới đây giải thích các thuật ngữ và chữ viết tắt được dùng thường xuyên trong báo cáo. Ví dụ chỉ minh họa cách hiểu trong phạm vi NASA C-MAPSS FD001; chúng không phải là kết quả thực nghiệm mới.

Thuật ngữ	Ý nghĩa và ví dụ trong báo cáo
Remaining Useful Life (RUL)	Tuổi thọ hữu ích còn lại, tính bằng số *cycle* trước khi động cơ hỏng. Với train, nhãn được tính bằng `max_cycle - cycle`; với test, báo cáo dự kiến tạo một dự đoán tại cycle cuối quan sát.
NASA C-MAPSS	Bộ dữ liệu mô phỏng các chuỗi vận hành đến khi hỏng của động cơ turbofan, được dùng làm nguồn dữ liệu chính cho báo cáo.
FD001	Tập con C-MAPSS được xét trong báo cáo, có một điều kiện vận hành và một dạng suy giảm High-Pressure Compressor (HPC).
Engine / `engine_id`	Một đơn vị động cơ trong dữ liệu. Engine là ranh giới của các phép ghép cặp, window và fold validation; hai split có thể dùng cùng giá trị ID nhưng không đại diện cho cùng một engine.
Cycle	Bước vận hành rời rạc trong lịch sử của một engine. Cycle không được quy đổi thành giờ hoặc thời gian lịch vì FD001 không cung cấp thông tin quy đổi.
Sensor	Kênh đo tín hiệu của động cơ. Mỗi quan sát FD001 có 21 sensor; các sensor được phân tích theo engine và không được xem là các biến độc lập hoàn toàn.
Operational settings	Ba trường mô tả điều kiện vận hành trong mỗi quan sát, được phân biệt với 21 sensor khi xây dựng contract và protocol thống kê.
Train / test	Train chứa lịch sử đầy đủ đến khi hỏng để tạo nhãn RUL; test chứa lịch sử bị cắt trước khi hỏng và có nhãn đánh giá tách riêng. Test không được dùng để chọn feature hoặc mô hình.
Data contract	Tập điều kiện hợp lệ cho dữ liệu, gồm schema, kiểu số, khóa duy nhất, split, cycle liên tục và các ràng buộc về nhãn. Input vi phạm contract bị từ chối, không tự động sửa raw.
Schema	Cấu trúc dữ liệu gồm tên cột, số cột, kiểu dữ liệu và quy ước khóa. Raw FD001 có 26 trường; reader bổ sung cột `split` cho bảng observations.
Data leakage	Rò rỉ dữ liệu xảy ra khi thông tin tương lai, test hoặc nhãn đánh giá đi vào feature, preprocessing, feature selection hoặc model selection. Ví dụ, window tại cycle 170 chỉ được dùng cycles 141–170.

Tiếp tục ở trang sau

Thuật ngữ	Ý nghĩa và ví dụ trong báo cáo
Exploratory Data Analysis (EDA)	Phân tích khám phá dữ liệu để kiểm tra phân phối, biến thiên và tín hiệu trước khi xây dựng feature. EDA hiện có trong repository chỉ đọc train.
Extract, Transform, Load (ETL)	Quy trình trích xuất, biến đổi và tải dữ liệu qua các lớp lưu trữ. Báo cáo dự kiến đối chiếu ETL bằng Pandas và PySpark, nhưng phần Silver/Gold chưa hoàn thành.
Bronze / Silver / Gold	Ba lớp dữ liệu dự kiến: Bronze giữ raw bất biến; Silver lưu observations đã kiểu hóa và validation; Gold lưu feature, prediction, metric và bảng phục vụ phân tích hoặc dashboard.
Window / trailing window	Đoạn quan sát liên tiếp gần nhất của một engine tại cycle đang xét. Cấu hình hiện tại dùng 30 cycles; window không đi qua ranh giới engine và không dùng quan sát tương lai.
Feature / feature engineering	Feature là biến đầu vào mô hình; feature engineering là quá trình tạo biến từ observations. Các feature dự kiến gồm last, mean, standard deviation, min, max và linear slope trong trailing window.
Baseline	Phương pháp tham chiếu tối giản để đánh giá ứng viên phức tạp hơn. Baseline hiện được đề xuất là dự đoán median RUL của nhãn training hợp lệ.
Metric	Đại lượng định lượng dùng để đánh giá mô hình hoặc dự báo. Báo cáo dự kiến dùng MAE, RMSE và NASA asymmetric scoring function; metric chưa có kết quả mô hình trong bản hiện tại.
Mean Absolute Error (MAE)	Trung bình của trị tuyệt đối sai số theo engine. MAE có đơn vị cycle và nhỏ hơn là tốt hơn.
Root Mean Squared Error (RMSE)	Căn bậc hai của trung bình bình phương sai số. RMSE cũng có đơn vị cycle và nhạy hơn MAE với sai số lớn.
NASA asymmetric scoring function	Hàm điểm phạt dự đoán sớm và muộn với mức khác nhau theo dấu của `prediction - truth`. Score nhỏ hơn là tốt hơn; đây không phải là ngưỡng bảo trì.
Paired analysis	Phân tích ghép hai phép đo của cùng một engine. Protocol M2 dùng mean của 30 cycles đầu và 30 cycles cuối, sau đó tính chênh lệch cuối trừ đầu.
Bootstrap	Lấy mẫu lại có hoàn lại để ước lượng độ bất định của một thống kê. Trong protocol M2, engine là đơn vị lấy mẫu và cặp đầu/cuối luôn được giữ cùng nhau.
Wilcoxon signed-rank test	Kiểm định phi tham số cho dữ liệu ghép cặp, sử dụng thứ hạng và dấu của chênh lệch. Kiểm định này không phải là kiểm định trực tiếp cho mean paired difference.

Tiếp tục ở trang sau

Thuật ngữ	Ý nghĩa và ví dụ trong báo cáo
p-value và adjusted *p-value*	*p-value* đo mức tương thích của dữ liệu với giả thuyết không theo protocol đã chọn. Adjusted *p-value* là giá trị sau hiệu chỉnh nhiều kiểm định; cả hai không được diễn giải thành độ lớn tác động.
Benjamini–Hochberg FDR	Benjamini–Hochberg False Discovery Rate kiểm soát tỷ lệ phát hiện sai kỳ vọng trong một họ kiểm định. Protocol M2 áp dụng cho đúng 21 sensor với $q = 0{,}05$; kết quả này không phải feature selection.
GroupKFold	Cách chia validation giữ toàn bộ mẫu của cùng một group trong một fold. Báo cáo dùng `engine_id` làm group để không đưa các window của cùng engine vào cả training và validation.
Principal Component Analysis (PCA)	Phương pháp tìm các hướng chiếu trực giao theo phương sai. PCA có thể tạo một ứng viên Health Index nhưng phải được fit trong training fold.
Health Index	Chỉ số vô hướng được suy ra từ các tín hiệu sensor để biểu diễn trạng thái sức khỏe tiềm năng. Trong báo cáo, Health Index là đầu vào cho ARIMA phụ, không phải nhãn RUL.
ARIMA	Mô hình chuỗi thời gian kết hợp autoregression, differencing và moving average. ARIMA được đề xuất cho forecast Health Index, không áp dụng trực tiếp lên countdown RUL xác định.
Parquet / SQLite	Parquet là định dạng bảng phục vụ lưu trữ phân tích; SQLite là lớp truy vấn và báo cáo cục bộ dự kiến trong Gold. Hai định dạng này chưa tạo thành pipeline Silver/Gold hoàn chỉnh trong bản hiện tại.
Spark / PySpark	Spark là framework xử lý dữ liệu phân tán; PySpark là API Python tương ứng. Báo cáo dự kiến dùng PySpark để tái hiện ETL và benchmark khả năng mở rộng, không mặc định rằng PySpark nhanh hơn Pandas trên FD001.

Danh mục hình

1 Luồng dữ liệu và trạng thái triển khai 14
2 Protocol thống kê đầu/cuối vòng đời 19
3 Phân phối tuổi thọ động cơ trên train 23
4 Phân phối correlation Spearman sensor–RUL theo động cơ 25
5 Phân phối chênh lệch sensor đầu/cuối theo engine 27
6 Chênh lệch cặp trung bình và khoảng tin cậy bootstrap 28

Danh mục bảng

2 Nhật ký công việc đến ngày 22/09/2026. 1
3 Công việc còn lại; thứ tự phụ thuộc, không phải lịch cam kết. 4
4 Quy mô FD001 đã ghi nhận trong báo cáo validation. 6
5 Data dictionary của FD001 trong repository. 6
6 Hồ sơ nhóm cột trên train FD001. 8
7 Profile 21 sensor trên train FD001. 8
8 Benchmark Pandas tạo engine summary 22
9 Rolling features M3.2 23
10 Thống kê tuổi thọ của 100 động cơ train 23
11 Correlation Spearman theo engine: các dòng minh họa 24
12 Ví dụ kết quả suy luận sensor đầu/cuối 26

1 Tiến độ thực hiện và kế hoạch tiếp theo

1.1 Cách ghi nhận tiến độ

Mỗi mốc ghi rõ công việc, lý do thực hiện, đầu ra có thể kiểm tra, trạng thái và phần còn lại. Trạng thái "đã có bằng chứng" chỉ áp dụng cho phạm vi đã được kiểm tra; việc một file tồn tại chưa đủ để kết luận toàn bộ mốc đã hoàn thành. Báo cáo được cập nhật cùng kết quả phân tích, thay vì đợi hoàn tất mô hình mới tổng hợp.

Bảng 2: Nhật ký công việc đến ngày 22/09/2026.

Mốc ghi nhận	Công việc	Lý do và bằng chứng	Trạng thái / còn lại
20–21/09/2026	Chuẩn bị FD001 và tài liệu nền tảng	Giữ một nguồn dữ liệu có lịch sử đến khi hỏng để xác định RUL. Đã có file raw, ghi chú provenance, danh mục tài liệu và phạm vi nghiên cứu.	Đã có; cần tiếp tục đọc và đối chiếu nguồn khi viết cơ sở lý thuyết.
22/09/2026	Kiểm tra dữ liệu và tạo train RUL	Kiểm tra trước phân tích để tránh tính toán trên schema hoặc khóa sai. JSON validation đạt; 20.631 train targets khớp 20.631 observations.	Đã có bằng chứng cho raw và train targets; chưa bao gồm rolling features.
22/09/2026	EDA mô tả chỉ dùng train	Tìm hiểu phân phối và tín hiệu trước khi xây dựng đặc trưng. Biên bản EDA ghi 69 tests đạt; notebook chạy từ kernel mới, 12 CSV và 11 PNG tái lập được.	Hoàn thành mốc EDA mô tả; riêng mốc này không gồm thống kê suy luận hoặc feature selection.
22/09/2026	Khởi tạo báo cáo LaTeX và nhật ký tiến độ	Lưu lập luận cùng bằng chứng ngay khi thực hiện để tránh mất lý do lựa chọn. Có nguồn theo phần, sơ đồ, bảng tiến độ và bản PDF.	Bản đang thực hiện; các kết quả mô hình và đánh giá cuối chưa có.
22/09/2026	Bổ sung cơ sở lý thuyết và protocol thống kê	Gắn nguồn cho phương pháp đã chọn; định nghĩa mean 30 cycles đầu/cuối, chênh lệch theo engine và giả định suy luận. Có phần lý thuyết, protocol và sơ đồ riêng.	Hoàn thành nội dung thiết kế của đợt này; kiểm định chưa chạy.

Mốc ghi nhận	Công việc	Lý do và bằng chứng	Trạng thái / còn lại
22/09/2026	Tạo contract dữ liệu cặp mốc 2	Hiện thực hóa helper chỉ nhận train, tạo một cặp 30 cycles đầu/cuối cho từng engine–sensor và kiểm tra khóa, ranh giới, thứ tự, số liệu. 17 tests tập trung và toàn bộ 86 tests đạt; test trên raw FD001 xác nhận 2.100 cặp trong bộ nhớ.	Tại thời điểm ghi nhận, mới có bằng chứng tạo cặp; artifact thống kê chưa được lưu.
22/09/2026	Chốt cấu hình thống kê M2.0	Người dùng chốt mean paired difference, percentile bootstrap 10.000 resamples, Wilcoxon permutation 100.000 resamples, BH FDR $q = 0{,}05$, seed 42 và quy tắc all-zero/NA trước khi xem kết quả suy luận.	Tại thời điểm ghi nhận, cấu hình đã chốt trong protocol; bảng, hình và p-value chưa được tạo.
22/09/2026	Hiện thực hóa helper suy luận M2.2	Thêm cấu hình M2.0, bootstrap mean CI, Wilcoxon permutation, BH FDR, validator bảng suy luận và fixtures tính tay. Focused tests đạt 24 tests; full suite đạt 93 tests. Raw FD001 được kiểm bằng cấu hình resample nhỏ trong bộ nhớ.	Có helper/validator; đã được dùng cho notebook M2.3a–M2.3b.
22/09/2026	Xuất artifact thống kê M2.3a	Notebook 02 chạy trên train, tạo 2.100 cặp engine–sensor, 21 dòng suy luận, `run_info.json` và `validation.json`. Validation ghi round-trip CSV đạt, raw hash không đổi, không đọc test.	Hoàn thành bảng/metadata; hình và diễn giải được bổ sung ở M2.3b.
22/09/2026	Hoàn thiện hình và diễn giải M2.3b	Notebook 02 tạo `difference_distributions.png` và `effect_size_ci.png` từ bảng M2.3a. Báo cáo bổ sung hình, giải thích sensor all-zero, `sensor_6`, `sensor_9` và giới hạn về feature selection.	Hoàn thành hình/diễn giải của mốc 2; bước còn lại tại thời điểm đó là M2.4 validation.

Mốc ghi nhận	Công việc	Lý do và bằng chứng	Trạng thái / còn lại
22/09/2026	Validation tái lập M2.4	Thêm runtime audit `STATISTICS_TRAIN_ONLY`, chạy focused statistics tests 26 tests, full suite 95 tests, notebook 02 hai lần từ fresh kernel và so sánh hai CSV cùng hai PNG theo SHA-256. Probe đọc `test_FD001.txt` và `RUL_FD001.txt` bị chặn; SHA-256 ba raw file không đổi.	Hoàn thành mốc thống kê train-only; chưa có ETL, rolling features, mô hình, ARIMA hoặc dashboard.
22/09/2026	M3.1 ETL Silver/Gold và parity	Pipeline đọc raw qua contract, ghi 33.727 observations và 20.631 train targets vào Silver Parquet, 200 engine summaries vào Gold Parquet/SQLite; round-trip đạt. Benchmark Pandas 1×/10×/100× với hai lần cold/warm ghi trong `outputs/etl/`. PySpark parity đạt cho ba bảng với `max_abs_diff=0`.	Hoàn thành ETL boundary và benchmark; rolling features chưa triển khai.
22/09/2026	M3.2 rolling features	Pipeline đọc Silver observations, tạo 27.927 rolling windows từ trailing window 30 cycles, gồm 17.731 train và 10.196 test rows. Mỗi row có 126 feature columns cho 21 sensors; tests tính tay kiểm tra sample std, slope, không dùng tương lai, không nối engine và round-trip Parquet.	Hoàn thành feature artifact; chưa có fold assignment, scaler, model fit, prediction hoặc metric.

1.2 Các mốc tiếp theo và điều kiện hoàn thành

Bảng 3: Công việc còn lại; thứ tự phụ thuộc, không phải lịch cam kết.

Công việc	Vì sao cần thực hiện	Bằng chứng cần bổ sung
Validation mô hình	Dùng feature artifact để tạo fold và đánh giá không rò rỉ dữ liệu.	Fold không trùng engine; scaler, feature selection, PCA và model chỉ fit training fold.
Mô hình và đánh giá cuối	Xác định khả năng dự đoán trên cùng protocol thay vì suy ra từ correlation.	Kết quả cross-validation, cấu hình được chọn, đúng 100 dự đoán test trước khi join nhãn; metric và khoảng tin cậy.
ARIMA và dashboard	Bổ sung phân tích chuỗi thời gian và cách đọc kết quả cảnh báo.	Forecast và diagnostics trên Health Index; dashboard kiểm tra được theo từng engine.
Hoàn thiện báo cáo	Liên kết kết quả, khó khăn và giới hạn với yêu cầu môn học.	PDF đủ citation/hình, demo và slide; kiểm tra tái lập toàn bộ.

Ví dụ cập nhật một mốc: khi ETL đã xuất Parquet nhưng chưa đối chiếu Pandas/PySpark, trạng thái vẫn là “đang thực hiện”. Chỉ khi có kết quả parity mới ghi hoàn thành phần tương đương dữ liệu; kết quả đó không đồng thời chứng minh PySpark nhanh hơn.

2 Bài toán, mục tiêu và phạm vi

2.1 Bài toán nghiên cứu

Đề tài phân tích chuỗi cảm biến động cơ nhằm ước lượng số cycle còn lại trước khi hỏng. Đầu vào tại cycle t là các quan sát đã có đến t của một động cơ; đầu ra là RUL dự đoán tại cycle đó. Supervised regression (hồi quy có giám sát) là cách mô hình hóa biến mục tiêu dạng số. Mục tiêu báo cáo là dự đoán RUL và giải thích bằng chứng cảnh báo sớm, không chỉ trình bày một thuật toán hồi quy.

Đối với train, đặt T_i là cycle cuối của động cơ i trong lịch sử đầy đủ đến khi hỏng. Nhãn được xác định bởi

$$y_{i,t} = T_i - t, \qquad 1 \leq t \leq T_i. \tag{1}$$

Ví dụ minh họa: một động cơ kết thúc ở cycle 200 có RUL bằng 30 cycles tại cycle 170 và bằng 0 tại cycle 200. Đây là ví dụ tính nhãn, không phải kết quả dự đoán.

Lịch sử test bị cắt trước khi hỏng nên không được lấy cycle cuối quan sát của test làm T_i. Mục tiêu đánh giá cuối là một dự đoán tại cycle cuối quan sát cho mỗi test engine. Bộ nhãn chính thức được tách khỏi quá trình lựa chọn mô hình.

2.2 Câu hỏi và giới hạn nghiên cứu

Các câu hỏi chính gồm: dữ liệu có đáp ứng contract không; cảm biến biến đổi thế nào theo vòng đời; đặc trưng từ 30 cycles gần nhất dự đoán RUL được đến mức nào; và kết quả có thể trình bày minh bạch ra sao. Bản hiện tại cung cấp bằng chứng cho câu hỏi chất lượng dữ liệu và một phần EDA, chưa trả lời được câu hỏi độ chính xác mô hình.

FD001 không có chi phí bảo trì, downtime hoặc lịch can thiệp. Do đó, RUL không tự động xác định thời điểm bảo trì tối ưu. Ngưỡng cảnh báo dự kiến 30 cycles chỉ dùng để minh họa cách đọc kết quả. Cycle cũng không được đổi thành giờ hoặc ngày vì chưa có thông tin quy đổi.

2.3 Liên hệ với yêu cầu môn học

Theo tài liệu môn học, tiêu chí quy trình xử lý dữ liệu chiếm 40%, trực quan hóa và phân tích 25%, chất lượng mã nguồn 20%, trình bày và phản biện 15%. Vì vậy, báo cáo dành nội dung riêng cho provenance, contract, ETL, lưu trữ và tái lập, bên cạnh mô hình dự đoán. Các trọng số này là tiêu chí chấm điểm, không phải tỷ lệ hoàn thành hiện tại.

Nguồn yêu cầu là hai tài liệu *Đề cương môn học Phân tích dữ liệu lớn* và *Danh sách đề tài báo cáo cuối kỳ*, được đối chiếu ngày 22/09/2026. Yêu cầu ghi rõ tiến độ được bổ sung theo trao đổi ngày này. Chưa có mẫu bìa bắt buộc hoặc tên giảng viên được xác nhận trong các nguồn đã đọc; bản hiện tại không tự điền các thông tin đó.

3 Dữ liệu và kiểm soát chất lượng

3.1 Nguồn dữ liệu và lý do lựa chọn

NASA C-MAPSS mô phỏng các chuỗi vận hành đến khi hỏng của động cơ turbofan. Nguồn dữ liệu được ghi nhận tại NASA Open Data [6]; tài liệu nền tảng đi kèm bộ dữ liệu là công trình của Saxena và cộng sự [9]. Mô tả đi kèm file raw xác định FD001 có một điều kiện vận hành và một dạng suy giảm High-Pressure Compressor (HPC, máy nén áp suất cao).

FD001 được chọn vì có lịch sử train đầy đủ đến khi hỏng, test bị cắt và nhãn đánh giá tách riêng, phù hợp với cách xây dựng bài toán RUL. Chỉ sử dụng một tập con giúp kiểm soát phạm vi thực nghiệm và làm rõ từng bước xử lý. Đổi lại, bằng chứng chỉ áp dụng trong điều kiện mô phỏng đang xét; không đủ để khẳng định khả năng khái quát trên động cơ thực tế hoặc điều kiện vận hành khác.

Bảng 4: Quy mô FD001 đã ghi nhận trong báo cáo validation.

Thành phần	Số dòng	Động cơ	Ý nghĩa
Train	20.631	100	Lịch sử đến khi hỏng
Test	13.096	100	Lịch sử bị cắt trước khi hỏng
Test labels	100	100	RUL tại cycle cuối quan sát

Mỗi dòng raw có 26 trường: engine ID, cycle, 3 operational settings và 21 sensors. Reader bổ sung cột `split`, tạo bảng observations có 27 cột. Khóa là bộ ba `(split, engine_id, cycle)`: train engine 1 và test engine 1 không phải cùng một thực thể. Nhãn test chỉ được kiểm tra cấu trúc riêng trong báo cáo integrity; việc này không phải đánh giá mô hình.

3.2 Từ điển dữ liệu và ý nghĩa cột

Dữ liệu train/test gốc không có header; tên cột trong báo cáo được đặt theo contract của repository. Bảng 5 mô tả vai trò từng nhóm cột và cách sử dụng trong phân tích. Nguồn NASA chỉ mô tả các trường sensor là "sensor measurement", không cung cấp tên vật lý, đơn vị đo hoặc mapping chi tiết cho từng sensor. Vì vậy, báo cáo giữ tên `sensor_1` đến `sensor_21`; không tự diễn giải một sensor cụ thể là nhiệt độ, áp suất hoặc tốc độ nếu chưa có nguồn xác nhận.

Bảng 5: Data dictionary của FD001 trong repository.

Cột hoặc nhóm cột	Ý nghĩa	Kiểu/miền đã kiểm tra	Vai trò trong báo cáo
`split`	Cột do reader bổ sung để phân biệt nguồn dữ liệu.	Chuỗi `train` hoặc `test`.	Giữ ranh giới train/test trong khóa dữ liệu và validation; không có trong raw text gốc.

Cột hoặc nhóm cột	Ý nghĩa	Kiểu/miền đã kiểm tra	Vai trò trong báo cáo
`engine_id`	Mã chuỗi động cơ trong từng split.	Số nguyên dương; FD001 có engine ID 1–100 ở cả train và test.	Đơn vị độc lập cho tuổi thọ, ghép cặp đầu/cuối, GroupKFold và đánh giá theo engine.
`cycle`	Thứ tự vận hành rời rạc của một engine.	Số nguyên dương, bắt đầu từ 1 và liên tục trong từng engine.	Trục thời gian rời rạc; dùng để tạo RUL train, window 30 cycles và phân tích đầu/cuối vòng đời.
`setting_1`–`setting_3`	Ba operational settings ảnh hưởng đến điều kiện vận hành.	`float64`; trên train, `setting_3` hằng ở 100.	Biến bối cảnh vận hành; không thuộc họ 21 sensor trong kiểm định M2.
`sensor_1`–`sensor_21`	Hai mươi mốt phép đo sensor trong mỗi cycle.	`float64` hữu hạn; không có missing trong train.	Nguồn tín hiệu chính cho EDA, paired analysis và feature engineering dự kiến.
`rul` trong train targets	Nhãn RUL được suy ra từ lịch sử train đầy đủ đến khi hỏng.	Số nguyên không âm; trên train hiện có miền 0–361 cycles.	Biến mục tiêu cho supervised regression; không phải cột raw.
`RUL_FD001.txt`	Nhãn RUL chính thức tại cycle cuối quan sát của mỗi engine test.	100 số nguyên không âm, ánh xạ dòng i sang test engine i.	Chỉ được ghép sau khi đã tạo 100 dự đoán test cuối cùng; không dùng trong EDA hoặc model selection.

Ví dụ đọc một dòng raw: dòng đầu tiên của `train_FD001.txt` có `engine_id=1`, `cycle=1`, ba settings lần lượt là $-0{,}0007$, $-0{,}0004$, $100{,}0$, sau đó là 21 giá trị sensor. Engine 1 trong train kết thúc ở cycle 192, nên sau khi tạo nhãn, dòng này có `rul=191`. Đây là nhãn hồi cứu của train, không phải dự đoán của mô hình.

3.3 Hồ sơ cột trên tập train

Bảng 6 tóm tắt các nhóm cột sau khi ghép RUL train. Các thống kê này lấy từ `outputs/eda/tables/input_summary.csv`, `outputs/eda/tables/rul_summary.csv` và `outputs/eda/tables/measurement_summary.csv`. Chúng dùng để hiểu cấu trúc dữ liệu trước khi xây feature; không phải bằng chứng lựa chọn mô hình.

Bảng 6: Hồ sơ nhóm cột trên train FD001.

Nhóm cột	Số cột	Missing	Nhận xét chính
Khóa `engine_id`, `cycle`	2	0	20.631 quan sát, 100 engine; không có khóa trùng, cycle liên tục trong từng engine.
Operational settings	3	0	`setting_1` có 158 giá trị khác nhau, `setting_2` có 13 giá trị, `setting_3` hằng bằng 100.
Sensors	21	0	Sáu sensor hằng toàn train; `sensor_6` chỉ có hai giá trị và hằng trong 38 engine.
Nhãn `rul` train	1	0	Miền 0–361 cycles; trung vị theo observation là 103 cycles.

Bảng 7: Profile 21 sensor trên train FD001.

Sensor	Số giá trị khác nhau	Min–max	Ghi chú
`sensor_1`	1	518,67–518,67	Hằng toàn train.
`sensor_2`	310	641,21–644,53	Biến thiên trên train.
`sensor_3`	3.012	1.571,04–1.616,91	Biến thiên trên train.
`sensor_4`	4.051	1.382,25–1.441,49	Biến thiên trên train; có chênh lệch đầu/cuối lớn trong M2.3a.
`sensor_5`	1	14,62–14,62	Hằng toàn train.
`sensor_6`	2	21,60–21,61	Biến thiên toàn train nhưng hằng trong 38/100 engine.
`sensor_7`	513	549,85–556,06	Biến thiên trên train.
`sensor_8`	53	2.387,90–2.388,56	Biến thiên thấp, nhưng không hằng.
`sensor_9`	6.403	9.021,73–9.244,59	Biến thiên mạnh; phân phối theo engine không đồng nhất hoàn toàn.
`sensor_10`	1	1,30–1,30	Hằng toàn train.
`sensor_11`	159	46,85–48,53	Biến thiên; Spearman median với RUL âm trong EDA.
`sensor_12`	427	518,69–523,38	Biến thiên; Spearman median với RUL dương trong EDA.
`sensor_13`	56	2.387,88–2.388,56	Biến thiên thấp, nhưng không hằng.
`sensor_14`	6.078	8.099,94–8.293,72	Biến thiên mạnh theo giá trị gốc.
`sensor_15`	1.918	8,3249–8,5848	Biến thiên trên train.
`sensor_16`	1	0,03–0,03	Hằng toàn train.
`sensor_17`	13	388–400	Biến thiên rời rạc trên train.
`sensor_18`	1	2.388–2.388	Hằng toàn train.
`sensor_19`	1	100–100	Hằng toàn train.
`sensor_20`	120	38,14–39,43	Biến thiên trên train.
`sensor_21`	4.745	22,8942–23,6184	Biến thiên trên train.

Các cột hằng toàn train không có correlation Spearman xác định với RUL và không thể tạo chênh lệch đầu/cuối khác 0 trong paired analysis. Tuy nhiên, báo cáo vẫn giữ chúng trong output thống kê để người đọc thấy rõ phạm vi kiểm định đủ 21 sensor. Với mô hình dự đoán, việc giữ hay loại sensor phải được quyết định trong từng training fold; không được dùng toàn bộ train để chọn sẵn feature.

3.4 Các kiểm tra đã triển khai và lý do

Reader và validator thực hiện kiểm tra số trường, schema, kiểu số hữu hạn, bảng không rỗng, split hợp lệ, khóa duy nhất và cycle liên tục trong từng động cơ. Profile FD001 đối chiếu thêm số dòng, ID động cơ và cycle bắt đầu. Với train targets, validator kiểm tra đủ khóa và đối chiếu Công thức 1.

Các kiểm tra được đặt trước EDA vì dữ liệu sai có thể tạo ra thống kê hợp lệ về mặt tính toán nhưng sai về ý nghĩa. Chẳng hạn, chuỗi cycle $1, 2, 4$ không được coi là ba bước liên tiếp khi xây window. Hai dòng cùng khóa cũng không được giữ lại như hai phép đo độc lập. Khi phát hiện lỗi, package báo `ContractError`; không tự điền dữ liệu, xóa dòng hoặc chỉnh raw để làm validation đạt.

3.5 Tính toàn vẹn và giới hạn của validation

Báo cáo JSON lưu SHA-256 của input để nhận biết phiên bản dữ liệu đã đọc. Dấu vân tay này chỉ hỗ trợ đối chiếu; tự tính checksum không chứng minh nguồn gốc file là chính xác. Việc kiểm tra row count và cycle continuity cũng không đủ phát hiện mọi thay đổi giá trị sensor.

Đặc biệt, một chuỗi train bị cắt đuôi nhưng vẫn liên tục có thể tạo ra RUL sai nếu bị hiểu nhầm là lịch sử đầy đủ. Vì vậy, giữ raw bất biến và ghi nguồn dữ liệu là điều kiện đi kèm công thức tính nhãn. API test labels được tách riêng nhưng chưa có cơ chế phân quyền filesystem ngăn mọi chương trình đọc file; ranh giới không dùng test để chọn mô hình còn phải được kiểm tra khi xây pipeline mô hình.

4 Cơ sở lý thuyết và nghiên cứu liên quan

Phần này giải thích các phương pháp thuộc phạm vi đã chọn. Công thức và kết quả từ tài liệu được phân biệt với cấu hình của báo cáo. Việc một phương pháp có cơ sở lý thuyết không đồng nghĩa phương pháp đó đã có kết quả trên FD001 trong repository.

4.1 Prognostics, RUL và phạm vi so sánh

Prognostics nghiên cứu diễn biến suy giảm và thời gian còn lại đến trạng thái hỏng. Với FD001, lịch sử train chạy đến khi hỏng cho phép tạo nhãn theo Công thức 1; test chỉ cung cấp một phần lịch sử. Nghiên cứu của Saxena và cộng sự mô tả mô phỏng suy giảm và bài toán đánh giá prognostics [9]. Tổng quan C-MAPSS của Ramasso và Saxena giúp phân biệt các tập con, điều kiện vận hành và protocol [7]. Vì vậy, cùng dùng tên C-MAPSS chưa đủ để so sánh trực tiếp metric giữa hai nghiên cứu.

Li và cộng sự nghiên cứu CNN với đầu vào chuỗi đa cảm biến [5]. Báo cáo sử dụng công trình này để tham khảo cách tạo window và ảnh hưởng của nhãn, không xem CNN là mô hình đã triển khai. Window 30 cycles là lựa chọn ban đầu đã chốt, chưa phải độ dài tối ưu. Nhãn chính của báo cáo là raw RUL; cap 125 chỉ thuộc phân tích độ nhạy được tách riêng. Kết quả dùng nhãn có cap hoặc loại bớt test engines không được xếp hạng trực tiếp với đánh giá raw RUL trên toàn bộ test.

Đánh giá prognostics còn quan tâm dự đoán xuất hiện sớm đến mức nào và thay đổi ra sao theo thời gian [8]. Một dự đoán cuối chuỗi cho mỗi test engine đo chất lượng tại thời điểm cắt; nó không thay thế đánh giá cả trajectory. Báo cáo chưa có chi phí can thiệp, nên ngưỡng cảnh báo 30 cycles chỉ minh họa cách sử dụng đầu ra, không phải lịch bảo trì tối ưu.

4.2 Biểu diễn chuỗi bằng trailing window

Gọi $\mathbf{x}_{i,t} \in \mathbb{R}^p$ là vector gồm p sensor được giữ của engine i tại cycle t. Với $w = 30$, một mẫu đầu vào chỉ chứa các quan sát hiện có:

$$W_{i,t} = [\mathbf{x}_{i,t-w+1}, \ldots, \mathbf{x}_{i,t}]^\mathsf{T} \in \mathbb{R}^{w \times p}, \qquad t \geq w. \tag{2}$$

Khác với cách đưa toàn bộ ma trận vào CNN [5], thiết kế hiện tại tổng hợp mỗi sensor thành sáu đặc trưng: last, mean, standard deviation, min, max và linear slope. Nếu giữ p sensor thì biểu diễn có $6p$ đặc trưng. Giá trị p phải được xác định trong training fold; không được nhìn toàn bộ train để chọn biến trước cross-validation.

Với các giá trị $z_1, \ldots, z_w$ của một sensor trong window, slope của hồi quy tuyến tính theo vị trí k là

$$b = \frac{\sum_{k=1}^{w}(k - \bar{k})(z_k - \bar{z})}{\sum_{k=1}^{w}(k - \bar{k})^2}. \tag{3}$$

Slope mô tả xu hướng tuyến tính cục bộ; mean và độ phân tán mô tả mức tín hiệu và biến động. Đây là diễn giải của đặc trưng, chưa chứng minh lợi ích dự đoán. Ví dụ, tại cycle 170, window gồm cycles 141–170; không dùng cycle 171 hoặc tuổi thọ đầy đủ của engine làm đầu vào.

4.3 Baseline và Ridge regression

Naive baseline trả về median của các nhãn training được protocol cho phép. Nó tạo mốc so sánh khi không sử dụng sensor. Cách lấy mẫu và trọng số nhãn phải được ghi cùng cấu hình, vì engine có nhiều window sẽ đóng góp nhiều nhãn nếu gộp trực tiếp.

Ridge dùng hàm dự đoán tuyến tính $\widehat{y} = \beta_0 + \mathbf{z}^\mathsf{T}\boldsymbol{\beta}$ với vector đặc trưng $\mathbf{z}$. Hàm mục tiêu gồm sai số bình phương và L2 regularization [13]:

$$\min_{\beta_0,\boldsymbol{\beta}} \sum_{n=1}^{N} (y_n - \beta_0 - \mathbf{z}_n^\mathsf{T}\boldsymbol{\beta})^2 + \lambda\|\boldsymbol{\beta}\|_2^2, \qquad \lambda \geq 0. \tag{4}$$

Trong biểu thức này intercept không bị phạt. Regularization hạn chế độ lớn hệ số, hữu ích khi các đặc trưng liên quan với nhau. Scaler cần fit trong training fold để các thang đo không chi phối mức phạt. Giá trị λ chưa được chọn; việc tối ưu loss bình phương cũng không đồng nghĩa MAE validation đã được tối ưu.

4.4 Histogram gradient boosting

Gradient boosting xây dựng hàm dự đoán bằng cách cộng dần các cây hồi quy, mỗi cây bổ sung thông tin để giảm loss của mô hình hiện tại. Một biểu diễn khái quát là

$$F_M(\mathbf{z}) = F_0(\mathbf{z}) + \eta \sum_{m=1}^{M} h_m(\mathbf{z}), \tag{5}$$

với h_m là cây thứ m, M là số vòng boosting và η là learning rate. HistGradientBoosting chia giá trị đặc trưng thành các bins để tìm phép tách trên histogram [12]. Mô hình có thể biểu diễn quan hệ phi tuyến và tương tác mà Ridge không mô tả trực tiếp. Số cây, độ phức tạp cây và learning rate còn phải chọn bằng validation; chưa có cấu hình hoặc kết quả để khẳng định mô hình này tốt hơn Ridge.

4.5 Đơn vị độc lập và đánh giá theo engine

Các window của cùng engine có thể chồng lấn và phụ thuộc thời gian. GroupKFold giữ một group ngoài training ở mỗi fold [10]; trong báo cáo, group là engine ID. Việc này đánh giá khả năng áp dụng cho engine chưa xuất hiện trong training, còn scaler, lựa chọn sensor và PCA đều phải học lại bên trong mỗi fold.

Với một dự đoán tại cycle cuối của mỗi test engine, đặt $e_i = \widehat{y}_i - y_i$ và m là số engine được đánh giá:

$$\text{MAE} = \frac{1}{m}\sum_{i=1}^{m}|e_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}e_i^2}. \tag{6}$$

Hai metric đều có đơn vị cycle và càng nhỏ càng tốt; RMSE nhạy hơn với sai số lớn. NASA asymmetric score bổ sung cách phạt khác nhau cho dự đoán sớm và muộn. Theo Công thức (5) trong Li và cộng sự [5], với cùng quy ước $e_i = \widehat{y}_i - y_i$:

$$S = \sum_{i=1}^{m} s(e_i), \qquad s(e) = \begin{cases} \exp(-e/13) - 1, & e < 0, \\ \exp(e/10) - 1, & e \geq 0. \end{cases} \tag{7}$$

Score càng nhỏ càng tốt; dự đoán RUL quá lớn ($e > 0$) bị phạt mạnh hơn dự đoán quá nhỏ với cùng độ lớn sai số. Công thức cộng score theo engine, không lấy trung bình; vì vậy số engine phải giống nhau khi đối chiếu. Khi triển khai cần kiểm thử cả hai dấu sai số theo quy ước này. Với nhiều dự đoán trên một validation engine, cách tổng hợp MAE được quy định tại Mục 5.

4.6 Paired analysis, bootstrap và nhiều kiểm định

Paired analysis ghép hai phép đo của cùng một engine để phân tích thay đổi trong vòng đời. Đại lượng cần ước lượng phải được nêu trước. Ví dụ, đại lượng đó có thể là mean paired difference (chênh lệch cặp trung bình) giữa cuối và đầu vòng đời. Đại lượng này biểu diễn mức thay đổi theo đơn vị gốc. P-value trả lời một câu hỏi khác: dữ liệu tương thích đến mức nào với giả thuyết không.

Wilcoxon signed-rank sử dụng thứ hạng của độ lớn các chênh lệch và dấu của chúng [18, 16]. Với giả định phân phối chênh lệch đối xứng quanh trung vị, kiểm định xem xét giả thuyết trung vị chênh lệch bằng 0. Các cặp cần độc lập giữa các engine. “Nonparametric” không có nghĩa là không cần giả định. Ties và chênh lệch bằng 0 ảnh hưởng đến phân phối thống kê và cách tính p-value. Không diễn giải Wilcoxon thành kiểm định trực tiếp trung bình chênh lệch.

Bootstrap ước lượng biến thiên của một thống kê bằng lấy mẫu lại có hoàn lại [3, 14]. Trong thiết kế này, engine là đơn vị lấy mẫu lại. Cặp đầu/cuối được giữ cùng nhau để bảo toàn quan hệ ghép cặp. Khoảng tin cậy 95% mô tả tính bao phủ qua các lần lấy mẫu theo quy trình và các giả định phù hợp; nó không phải xác suất 95% rằng một tham số cố định nằm trong khoảng đã tính. Không coi các cycles liên tiếp là các đơn vị độc lập để tạo khoảng tin cậy hẹp hơn.

Với nhiều sensor, Benjamini–Hochberg kiểm soát False Discovery Rate (FDR, tỷ lệ phát hiện sai kỳ vọng) dưới những điều kiện phù hợp [2, 15]. Nếu sắp K p-values thành $p_{(1)} \leq \cdots \leq p_{(K)}$, quy tắc step-up tìm chỉ số lớn nhất k thỏa $p_{(k)} \leq kq/K$, với q là mức FDR đã định trước. Bảo đảm này không áp dụng cho mọi cấu trúc phụ thuộc. Vì vậy, cần thảo luận tương quan giữa các sensor khi diễn giải. Khoảng tin cậy riêng từng sensor không tự trở thành khoảng tin cậy đồng thời sau hiệu chỉnh p-value.

4.7 Health Index từ PCA và phân tích ARIMA

PCA tìm các phương chiếu trực giao có phương sai lớn từ dữ liệu đã được xử lý [11]. Nếu dùng thành phần đầu làm Health Index thì $h_{i,t} = \mathbf{v}_1^{\mathsf{T}}\widetilde{\mathbf{x}}_{i,t}$, với $\mathbf{v}_1$ và phép chuẩn hóa được học từ training fold. Đây là cách xây dựng một ứng viên Health Index. Số thành phần và quy ước chiều tăng/giảm vẫn cần được đặc tả. Phương sai lớn không tự chứng minh ý nghĩa sức khỏe hoặc tính đơn điệu. Dấu của vector riêng cũng không xác định ý nghĩa suy giảm.

ARIMA(p, d, q) kết hợp autoregression bậc p, sai phân bậc d và moving average bậc q [4]. Viết $u_t = (1 - B)^d h_t$, trong đó $Bh_t = h_{t-1}$, một dạng mô hình là

$$u_t = c + \sum_{j=1}^{p} \phi_j u_{t-j} + \varepsilon_t + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j}. \tag{8}$$

Sai phân hỗ trợ xử lý tính không dừng, nhưng không nên tăng bậc tùy ý. ADF kiểm định giả thuyết có unit root [17]. Ljung–Box khảo sát autocorrelation của residual tại các lag được chọn [4]. Không bác bỏ giả thuyết không không chứng minh residual độc lập. Forecast MAE/RMSE và coverage của prediction interval cần được đánh giá trên đoạn chuỗi chưa dùng để fit. Coverage là tỷ lệ quan sát nằm trong khoảng dự đoán; đại lượng này khác khoảng tin cậy của một metric tổng hợp.

4.8 Spark, lưu trữ và giới hạn của benchmark

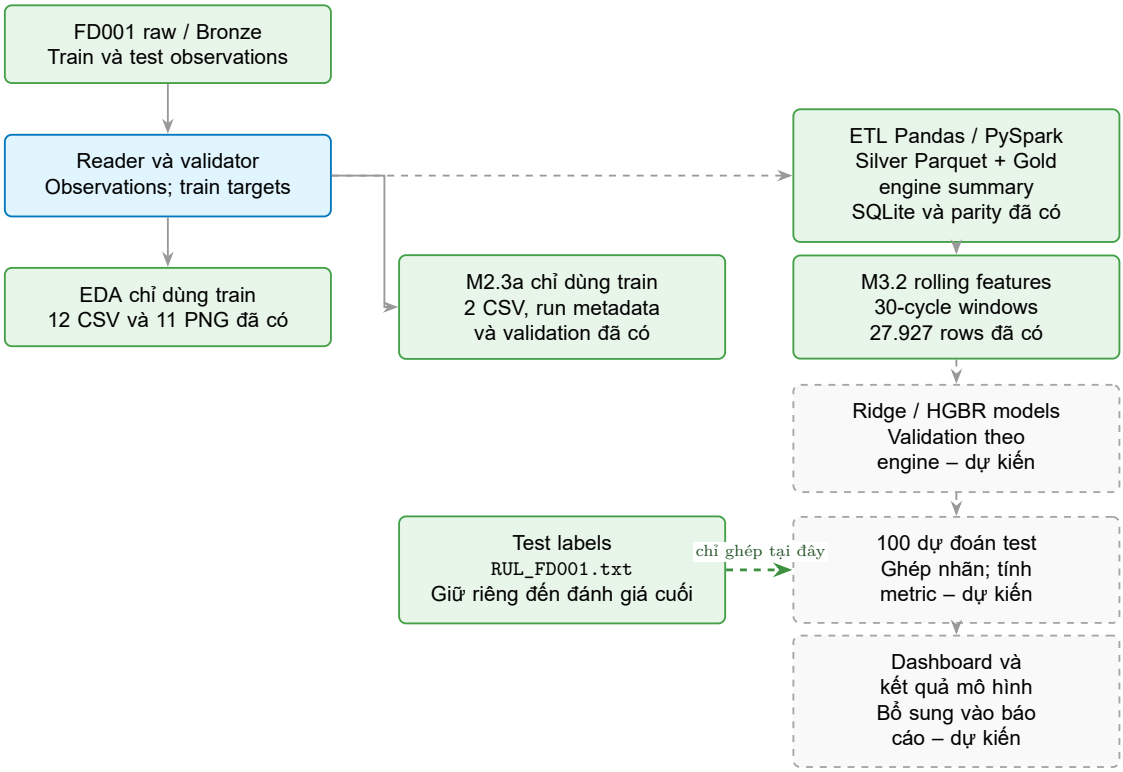
RDD là nền tảng trừu tượng của Spark cho xử lý dữ liệu phân tán và khả năng phục hồi tính toán [19]. Với dữ liệu có schema, Spark SQL/DataFrame cung cấp biểu diễn và các phép xử lý có cấu trúc [1]. Báo cáo dùng nguồn này cho phần ETL; không suy từ lý thuyết phân tán rằng Spark chắc chắn nhanh hơn Pandas trên FD001.

Bronze/Silver/Gold là cách tổ chức dữ liệu của báo cáo. Parquet phục vụ lưu bảng phân tích; SQLite phục vụ truy vấn và trình bày local. Bằng chứng cần có là khả năng đọc lại artifact, tính tương đương khóa/giá trị và thời gian đo theo cấu hình công bố. Dữ liệu nhân bản chỉ dùng benchmark xử lý, không tăng số mẫu khoa học độc lập. Streaming và LSTM vẫn là hướng đọc bổ sung, không mở rộng phạm vi thực nghiệm đã chốt.

5 Thiết kế phương pháp và lý do lựa chọn

5.1 Luồng xử lý và trạng thái triển khai

Hình 1 phân biệt các nhánh đã hiện thực hóa với nhánh dự kiến. Dữ liệu raw là đầu vào bất biến. Reader và validator tạo quan sát cùng nhãn RUL train trong bộ nhớ; EDA và suy luận M2.3a chỉ dùng train, đồng thời đã xuất bảng, hình hoặc metadata tương ứng. M3.1 đã ghi observations và train targets vào Silver Parquet, đồng thời tạo Gold engine summary ở Parquet và SQLite. M3.2 đã tạo Gold rolling features từ Silver observations.



Hình 1: Luồng dữ liệu: đường liền thể hiện nhánh đã hiện thực hóa hoặc đã có artifact; đường và khung nét đứt thể hiện phần dự kiến. Nhãn test được vẽ như side input tách riêng và chỉ ghép vào bước đánh giá cuối.

Bronze giữ dữ liệu nguồn để truy vết. Silver lưu observations đã gán kiểu dữ liệu và train targets trong Parquet; Gold M3.1 lưu engine summary trong Parquet/SQLite để kiểm tra boundary và truy vấn cục bộ. Gold M3.2 lưu rolling features trong Parquet, tách khỏi bước fit scaler hoặc mô hình. Cách tách lớp giúp phân biệt dữ liệu gốc với dữ liệu suy ra và xác định bước gây mất hoặc nhân bản dòng. Pandas là reference implementation; PySpark tái hiện ba boundary và parity checker đã xác nhận ba bảng khớp với sai khác lớn nhất bằng 0 trong lần chạy local có JRE. Chưa có bằng chứng để kết luận công cụ nào nhanh hơn.

FD001 có dung lượng nhỏ. Benchmark M3.1 ở mức $1\times$, $10\times$ và $100\times$ chỉ đánh giá chi phí tạo engine summary; bản nhân dữ liệu đổi engine ID và tách khỏi phân tích khoa học. Việc nhân

bản quan sát không tạo thêm động cơ độc lập để huấn luyện hoặc thu hẹp khoảng tin cậy.

5.2 EDA theo đơn vị động cơ

EDA chỉ sử dụng train để tìm hiểu dữ liệu mà không dựa vào test khi quyết định phương pháp. Phân phối tuổi thọ tính mỗi động cơ một lần. Ngược lại, histogram RUL theo quan sát cho động cơ sống lâu nhiều quan sát hơn. Hai cách tổng hợp trả lời hai câu hỏi khác nhau và không được diễn giải thay thế nhau.

Correlation giữa sensor và RUL được tính bằng Spearman riêng trong từng động cơ, sau đó tóm tắt phân phối giữa các động cơ. Lựa chọn này khảo sát liên hệ đơn điệu và tránh để số dòng của một động cơ quyết định trực tiếp trọng số trong bảng tổng hợp correlation. Tuy vậy, dependence theo thời gian vẫn tồn tại; correlation mô tả chưa chứng minh quan hệ nhân quả hoặc hiệu quả dự đoán ngoài mẫu.

Sensor hằng có correlation không xác định, được giữ dưới dạng NaN thay vì gán 0. Ba động cơ minh họa được chọn gần các tứ phân vị tuổi thọ; nếu bằng khoảng cách, chọn ID nhỏ hơn. Quy tắc này dựa trên tuổi thọ, không dựa trên quỹ đạo sensor có hình dạng dễ quan sát. Quỹ đạo hiện có biểu diễn giá trị gốc theo cycle, không smooth; mỗi ô dùng thang đo riêng. Ba động cơ này không đại diện thống kê. Tuổi thọ đầy đủ chỉ dùng cho phân tích hồi cứu sau khi đã quan sát cả chuỗi, không được dùng làm feature dự đoán tại cycle hiện tại.

5.3 Đặc trưng từ window M3.2

Biểu diễn chính đã triển khai là trailing window (cửa sổ chỉ chứa hiện tại và quá khứ) dài 30 cycles. Ví dụ, dự đoán tại cycle 170 dùng các cycle 141–170 của cùng động cơ. Mỗi output row khóa bởi `(split, engine_id, cycle)`, trong đó `cycle` là cycle cuối của window. Transformer từ chối engine có ít hơn 30 cycles; không padding, không impute và không rút ngắn window.

M3.2 giữ đủ 21 sensors trong artifact để chưa thực hiện feature selection trên toàn bộ train. Với mỗi sensor, pipeline tạo sáu đặc trưng theo mẫu `<sensor>__<stat>`: giá trị cuối, mean, sample standard deviation (`ddof=1`), min, max và linear slope theo vị trí 1–30 trong window. Nếu một sensor bằng đúng số cycle trong window 1–30 thì các feature tương ứng là `last=30`, `mean=15,5`, `min=1`, `max=30` và `slope=1`. Sample standard deviation dùng quy ước thống kê mẫu để nhất quán với cách mô tả biến thiên trong EDA, nhưng quyết định này được ghi riêng trong `outputs/features/run_info.json` thay vì suy ngầm từ default.

Artifact hiện có 27.927 rows, gồm 17.731 train windows và 10.196 test windows. Bước này không đọc `data/raw/RUL_FD001.txt`, không join nhãn test, không fit scaler, không chọn sensor và không huấn luyện mô hình. Các lựa chọn phụ thuộc dữ liệu như loại sensor hằng, PCA hoặc scaler vẫn phải học trong training fold ở mốc mô hình.

5.4 Mô hình và đánh giá: phần dự kiến

Naive baseline theo trung vị RUL cung cấp mốc so sánh khi không dùng tín hiệu sensor. Ridge là baseline tuyến tính có regularization; HistGradientBoostingRegressor là ứng viên phi tuyến. So sánh này nhằm đánh giá lợi ích của tín hiệu sensor so với dự đoán đơn giản và mức cải thiện khi cho phép quan hệ phi tuyến. Chưa có hyperparameter search hoặc kết quả để xếp hạng ba phương pháp.

Model selection dự kiến dùng GroupKFold 5-fold theo `engine_id`. Các window liền nhau của một động cơ có thể dùng chung phần lớn quan sát: hai window 141–170 và 142–171 dùng chung 29 cycles. Nếu chia ngẫu nhiên theo dòng, những window gần như giống nhau có thể nằm ở cả train và validation, gây data leakage (rò rỉ dữ liệu). Vì vậy, toàn bộ window của một động cơ phải thuộc cùng fold; scaler, lựa chọn sensor và mô hình chỉ fit trên training fold.

Metric chọn mô hình dự kiến là mean engine-level MAE. Với m động cơ validation và n_i dự đoán được đánh giá của động cơ i, đặt

$$\mathrm{MAE}_{\text{engine}} = \frac{1}{m}\sum_{i=1}^{m}\left[\frac{1}{n_i}\sum_{t\in\mathcal{E}_i}|y_{i,t}-\widehat{y}_{i,t}|\right], \tag{9}$$

trong đó $\mathcal{E}_i$ là tập cycle được protocol chọn để đánh giá. Cách lấy trung bình này cho mỗi động cơ trọng số bằng nhau. Quy tắc chọn $\mathcal{E}_i$ trong cross-validation còn phải được cụ thể hóa và lưu cùng cấu hình trước khi huấn luyện. RMSE và NASA asymmetric score dự kiến là metric phụ; chưa có số liệu cho các metric này.

Sau khi chọn cấu hình bằng train/validation, mô hình được refit trên toàn train và tạo một dự đoán cho mỗi động cơ test tại cycle cuối quan sát. Chỉ sau đó mới ghép nhãn test để đánh giá cuối. Khoảng tin cậy dự kiến dùng bootstrap theo động cơ nhằm giữ nguyên đơn vị chuỗi, thay vì giả định các window chồng lấn là mẫu độc lập.

5.5 Thống kê hiện có và ARIMA dự kiến

M2.3a–M2.3b so sánh trung bình 30 cycles đầu và 30 cycles cuối trong cùng động cơ, theo quyết định ngày 22/09/2026. Helper tạo cặp và các hàm suy luận theo sensor đã được hiện thực hóa, kiểm tra bằng fixture và chỉ xử lý train. Notebook 02 đã xuất bảng cặp, bảng suy luận, metadata, validation và hai hình panel tại `outputs/statistics/`; bảng cặp có 2.100 dòng và bảng suy luận có 21 dòng. Mục 6 định nghĩa cặp, chiều chênh lệch, đầu ra, giả định và cấu hình M2.0 đã chốt. Mục 4 trình bày cơ sở của chênh lệch cặp trung bình, khoảng tin cậy bootstrap 95%, Wilcoxon và Benjamini–Hochberg FDR. Hình phân phối và effect size được tạo theo panel riêng cho từng sensor, kèm diễn giải về sensor hằng, dấu chênh lệch và heterogeneity giữa các engine. Kết quả M2.3a–M2.3b không phải feature selection và không kết luận về hiệu quả dự đoán RUL.

ARIMA là phân tích phụ trên Health Index từ PCA được fit trong training fold. ARIMA không áp dụng trực tiếp lên nhãn RUL train vì Công thức 1 tạo một countdown tất định; dự báo

lại countdown không chứng minh khả năng khai thác tín hiệu sensor. Kế hoạch hiện tại chọn 10 động cơ validation đại diện, dùng 70% đầu chuỗi để fit và 30% cuối để đánh giá. Cách chọn động cơ cụ thể và ARIMA order còn chưa triển khai. Kết quả sẽ cần ADF, Ljung–Box, forecast MAE/RMSE và coverage của khoảng dự đoán 95%.

6 Protocol thống kê suy luận mốc 2

6.1 Mục tiêu, phạm vi và trạng thái quyết định

Mục tiêu là đánh giá mức thay đổi của sensor giữa đầu và cuối vòng đời trong cùng engine trên tập train FD001. Ngày 22/09/2026 đã chốt dùng trung bình **30 cycles đầu và 30 cycles cuối**. Phân tích chỉ đọc train và dùng engine làm đơn vị ghép cặp. Đây là protocol trước khi chạy kiểm định, được viết sau EDA; các giả thuyết phát sinh từ EDA mang tính khám phá, không được mô tả là đã đăng ký trước khi xem dữ liệu.

Phân tích ghép cặp (paired analysis), chênh lệch cặp trung bình, bootstrap 95%, Wilcoxon khi phù hợp, Benjamini–Hochberg và seed 42 thuộc cấu hình hiện có. Cấu hình M2.0 đã được chốt trước khi chạy suy luận: dùng mean paired difference, percentile bootstrap, Wilcoxon permutation và BH trên họ 21 sensors. Notebook M2.3a–M2.3b đã chạy theo cấu hình này, đồng thời xuất bảng, metadata và hai hình panel train-only.

6.2 Định nghĩa cặp đầu/cuối và ví dụ

Với $x_{i,t,s}$ là giá trị sensor s của engine i ở cycle t, và T_i là cycle cuối của train engine đó, đặt

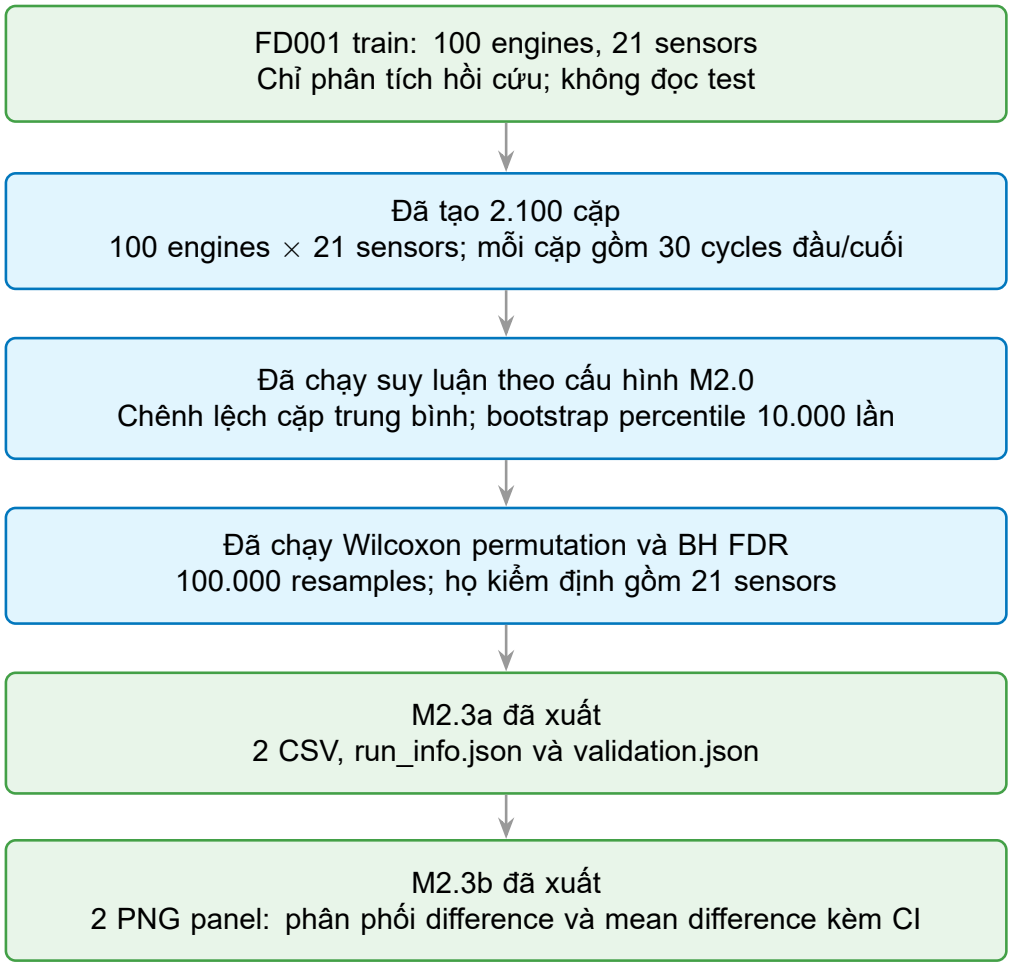
$$E_{i,s} = \frac{1}{30}\sum_{t=1}^{30} x_{i,t,s}, \qquad L_{i,s} = \frac{1}{30}\sum_{t=T_i-29}^{T_i} x_{i,t,s}, \tag{10}$$

$$D_{i,s} = L_{i,s} - E_{i,s}. \tag{11}$$

$D_{i,s} > 0$ biểu thị mức trung bình cuối đời lớn hơn đầu đời. Mỗi engine đóng góp một cặp cho mỗi sensor, bất kể tuổi thọ. Hai đoạn phải đủ dữ liệu, liên tục và không chồng lấn. Tuổi thọ train nhỏ nhất đã ghi nhận là 128 cycles, nên độ dài hiện tại đáp ứng điều kiện này; bước triển khai vẫn phải kiểm tra lại input contract.

Ví dụ tính chỉ số: với engine có $T_i = 200$, đoạn đầu là cycles 1–30 và đoạn cuối là cycles 171–200. Không nhầm đoạn cuối đời này với window 141–170 dùng dự đoán tại cycle 170. T_i được biết khi phân tích hồi cứu train; không được dùng để xác định đặc trưng ở thời điểm dự đoán trên test.

Giữ độ dài bằng nhau giúp so sánh hai mức tín hiệu trên cùng số cycles. Tuy nhiên, 30 cycles chiếm tỷ lệ vòng đời khác nhau giữa các engine. Protocol này trả lời câu hỏi về hai đoạn có độ dài tuyệt đối cố định, không phải so sánh cùng phần trăm tuổi thọ và cũng không tự xác định chính xác thời điểm bắt đầu suy giảm.



Hình 2: Thiết kế phân tích mốc 2; bước tạo cặp, suy luận và xuất bảng, metadata, validation và hình M2.3a–M2.3b đã có helper, notebook và validation.

6.3 Đại lượng cần báo cáo và điều kiện diễn giải

Bảng kết quả M2.3a giữ đủ 21 sensors, không chọn danh sách theo thứ hạng correlation của EDA. Ba operational settings không được gộp vào cùng họ giả thuyết sensor. Bảng ghi số cặp hợp lệ, số chênh lệch bằng 0, chênh lệch cặp trung bình, khoảng tin cậy, thống kê kiểm định, p-value gốc, p-value hiệu chỉnh và trạng thái có thể kiểm định.

Dùng chênh lệch cặp trung bình $\widehat{\Delta}_s = m^{-1} \sum_i D_{i,s}$ làm đại lượng chính theo đơn vị gốc, kèm trung vị và phân phối $D_{i,s}$. Đại lượng này mô tả mức thay đổi, không phải đại lượng thay đổi chuẩn hóa. Vì các sensor có thang đo khác nhau, không so sánh trực tiếp độ lớn của chúng. Không gán đơn vị vật lý nếu nguồn chưa xác nhận. Bootstrap lấy mẫu lại các engine cùng cặp E, L để tạo khoảng tin cậy 95% cho $\widehat{\Delta}_s$ [14]. Cấu hình M2.0 dùng 10.000 resamples và percentile interval.

Wilcoxon hai phía kiểm định giả thuyết trung vị của chênh lệch bằng 0. Việc diễn giải dựa vào giả định phân phối chênh lệch đối xứng quanh trung vị và các cặp độc lập giữa engine [16].

Cần xem phân phối chênh lệch và ghi rõ giới hạn của giả định đối xứng; bootstrap của trung bình không thay thế giả định này. Sensor hằng hoặc có toàn bộ $D_{i,s} = 0$ vẫn phải xuất hiện trong bảng dưới trạng thái không có biến thiên quan sát được. Không biến p-value không xác định thành 0 hoặc tự kết luận sensor có giá trị dự đoán bằng 0.

Benjamini–Hochberg áp dụng cho một họ kiểm định đã xác định trước [2, 15]. Cấu hình M2.0 giữ họ gồm 21 sensors, dùng giá trị 1 chỉ trong vector hiệu chỉnh cho trường hợp không kiểm định được, đồng thời giữ p-value gốc là không áp dụng và ghi rõ trạng thái trong bảng. Quy ước này tránh thu nhỏ họ giả thuyết theo kết quả thuận lợi; nó không sửa được p-value thiếu hợp lệ hoặc bảo đảm FDR dưới mọi dạng phụ thuộc giữa sensors. Các khoảng tin cậy bootstrap riêng lẻ không phải simultaneous intervals.

6.4 Cấu hình M2.0 đã chốt trước triển khai

Các giá trị sau là cấu hình đã chốt ngày 22/09/2026 trước khi tính kết quả; không lấy default của thư viện làm quyết định ngầm.

- Đại lượng chính: mean paired difference; kèm median, số dấu dương, số dấu âm và số zero để xem phân phối lệch. Chưa bổ sung đại lượng thay đổi chuẩn hóa vì chưa có câu hỏi yêu cầu so sánh giữa sensors.
- Bootstrap: 10.000 lần lấy mẫu lại 100 engines có hoàn lại, seed 42; dùng percentile interval tại các quantile 2,5% và 97,5%. Ghi rõ giới hạn độ bao phủ hữu hạn; không mặc định tương đương BCa. Trường hợp phân phối bootstrap suy biến phải gắn nhãn, không diễn giải khoảng bằng 0 là chắc chắn tuyệt đối.
- Wilcoxon: hai phía, zero method `wilcox`, không dùng continuity correction và không dùng `auto`. Dùng permutation với 100.000 resamples và RNG cố định cho sensors có thể kiểm định. Zero được xác định bằng `difference == 0.0` sau tính mean `float64`; không tự làm tròn bằng một ngưỡng tùy ý cho mọi sensor.
- All-zero và p-value thiếu: nếu toàn bộ $D_{i,s} = 0$, giữ sensor trong bảng với trạng thái all-zero, `raw_p=NA` và không diễn giải như một kiểm định thành công. Lỗi số bất ngờ phải làm lần chạy dừng thay vì biến thành p-value hợp lệ.
- FDR: dùng Benjamini–Hochberg với $q = 0{,}05$ trên đúng họ 21 sensors bằng hàm SciPy đã cố định trong metadata. Với p-value không áp dụng, chỉ dùng sentinel 1,0 bên trong vector hiệu chỉnh; output vẫn giữ `raw_p=NA` và trạng thái riêng.
- RNG và metadata: dùng seed 42 nhưng tách luồng RNG theo sensor và mục đích bootstrap/permutation để đổi thứ tự chạy không đổi kết quả của cùng sensor. `run_info.json` phải ghi cấu hình thực tế, phiên bản SciPy/NumPy/Pandas, data hash và commit hash.

Cấu hình này khóa cách tính trước khi xem p-value hoặc khoảng tin cậy thật. Nếu dùng phương án khác ở lượt sau, phải cập nhật protocol, lý do và thời điểm thay đổi trước khi tạo lại bảng kết quả; không trình bày lựa chọn mới như đã xác định từ đầu.

6.5 Đầu ra và điều kiện nghiệm thu

M2.3a đã xuất bảng cặp theo engine/sensor tại `engine_sensor_pairs.csv`, bảng suy luận theo sensor tại `sensor_inference.csv`, cùng `run_info.json` và `validation.json`. M2.3b đã bổ sung `difference_distributions.png`, `effect_size_ci.png` và diễn giải theo engine; validation xác nhận hai bảng round-trip, raw train không đổi và lần chạy không dùng quan sát test hoặc nhãn test. Artifact hiện có không thay thế kết quả mô hình RUL chưa chạy.

Kiểm tra phải bao gồm chỉ số 30 cycles đúng trong từng engine, không chồng đoạn, ghép cặp một–một, ví dụ tính tay và trường hợp hằng/ties/zeros. Bootstrap phải lấy mẫu theo engine và giữ cặp; FDR cần được kiểm tra trên một vector p-value nhỏ có đáp án biết trước. Chạy lại với cùng input và seed phải tái lập kết quả; raw và EDA gốc giữ nguyên. Khi có hình và diễn giải mới, tiếp tục cập nhật phần kết quả, nhật ký và bằng chứng kiểm tra.

Kết luận thống kê chỉ áp dụng cho thay đổi trên train mô phỏng theo protocol này. Phát hiện thay đổi không chứng minh quan hệ nhân quả hoặc hiệu quả dự đoán ngoài mẫu. Lựa chọn sensor cho mô hình vẫn phải học trong training fold; không dùng danh sách có ý nghĩa thống kê trên toàn train để chọn feature trước cross-validation.

7 Kết quả hiện có và diễn giải

7.1 Kết quả validation

File `outputs/validation/fd001.json` ghi trạng thái `passed` cho quan sát train, nhãn RUL train và quan sát test. Số dòng trước/sau khi tạo nhãn RUL train đều là 20.631. Luồng mặc định ghi `test_labels_requested=false` và `test_labels_joined=false`. Báo cáo integrity riêng ghi nhận 100 nhãn hợp lệ về cấu trúc, không tính metric dự đoán.

Các kết quả này cho phép tiếp tục phân tích trên dữ liệu đạt contract đã định nghĩa. Chúng không chứng minh dữ liệu không có nhiễu, không có sai lệch mô phỏng hoặc mô hình sẽ dự đoán chính xác.

7.2 ETL Silver/Gold và benchmark

M3.1 chạy pipeline Pandas và PySpark từ `data/raw/` sau khi đọc và kiểm tra contract. Silver gồm 33.727 observations của train và test trong `silver/observations.parquet`, cùng 20.631 train targets trong `silver/train_targets.parquet`. Gold gồm 200 dòng engine summary trong `gold/engine_summary.parquet` và bảng truy vấn tương ứng trong `gold/etl.sqlite`. Các artifact được đọc lại sau khi ghi; checksum raw được lưu trong `run_info.json`. Engine summary không phải rolling feature và chưa được dùng để huấn luyện mô hình.

Bảng 8: Benchmark Pandas tạo engine summary trên observations nhân bản trong bộ nhớ; không dùng cho huấn luyện.

Scale	Input rows	Engine rows	Cold (s)	Warm (s)
1×	33.727	200	0,0195	0,0196
10×	337.270	2.000	0,1589	0,1474
100×	3.372.700	20.000	1,6752	1,6732

Lần đo dùng Python 3.12.14, Pandas 2.3.3 trên Windows 11, mỗi scale hai lần đo cold/warm theo metadata. Đây là chi phí của một phép tổng hợp cục bộ, không phải bằng chứng về xử lý phân tán. `parity.json` ghi Pandas/PySpark khớp schema, khóa, số dòng và giá trị số cho ba bảng Silver/Gold; cả ba `max_abs_diff` bằng 0,0. Lần chạy Spark dùng JRE portable trong `tmp/` và không cấu hình HDFS hoặc cluster.

7.3 Rolling features M3.2

M3.2 đọc `outputs/etl/silver/observations.parquet` và tạo `outputs/features/gold/rolling_features.parquet`. Artifact có 27.927 rows: 17.731 train windows và 10.196 test windows. Mỗi row có khóa `(split, engine_id, cycle)` và 126 feature columns cho 21 sensors nhân sáu thống kê: `last`, `mean`, sample `std` với `ddof=1`, `min`, `max` và `slope`. Tổng số cột là 129 khi tính cả ba khóa.

Bảng 9: Số dòng rolling features M3.2 theo split. Mỗi window dài 30 cycles và kết thúc tại khóa observation của row.

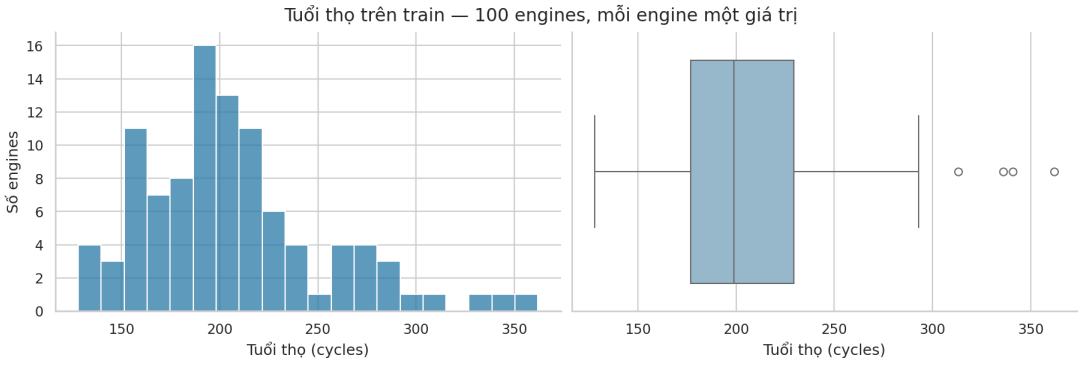
Split	Engines	Rolling windows
Train	100	17.731
Test	100	10.196
Tổng	200	27.927

Validator kiểm tra schema, khóa duy nhất, thứ tự canonical, số hữu hạn và các cycle có đủ trailing window trong cùng engine/split. Tests tính tay xác nhận sensor bằng cycle có `last`, mean, sample standard deviation, min, max và slope đúng; thay đổi cycle 31 không làm đổi feature tại cycle 30; hai engine không bị nối window. Metadata ghi `joins_test_labels=false` và `modeling.status=not_started`; vì vậy artifact này chưa phải kết quả mô hình hoặc metric.

7.4 Tuổi thọ động cơ trên train

Bảng 10: Thống kê tuổi thọ của 100 động cơ train, đơn vị cycle. Nguồn: `lifetime_summary.csv`.

Thống kê	Giá trị	Thống kê	Giá trị
Trung bình	206,31	Độ lệch chuẩn mẫu	46,34
Nhỏ nhất	128	Lớn nhất	362
Tứ phân vị 25%	177	Trung vị	199
Tứ phân vị 75%	229,25	Số động cơ	100



Hình 3: Phân phối tuổi thọ train; mỗi động cơ đóng góp một giá trị. Hình lấy từ artifact EDA hiện có.

Tuổi thọ quan sát dao động từ 128 đến 362 cycles; trung bình lớn hơn trung vị. Sự khác biệt về độ dài chuỗi cho thấy cách lấy trung bình theo quan sát và theo động cơ có thể tạo trọng số khác nhau. Đây là cơ sở để ghi rõ đơn vị tổng hợp và dùng động cơ làm đơn vị đánh giá dự kiến. Các con số không được diễn giải thành tuổi thọ theo giờ hoặc ngày.

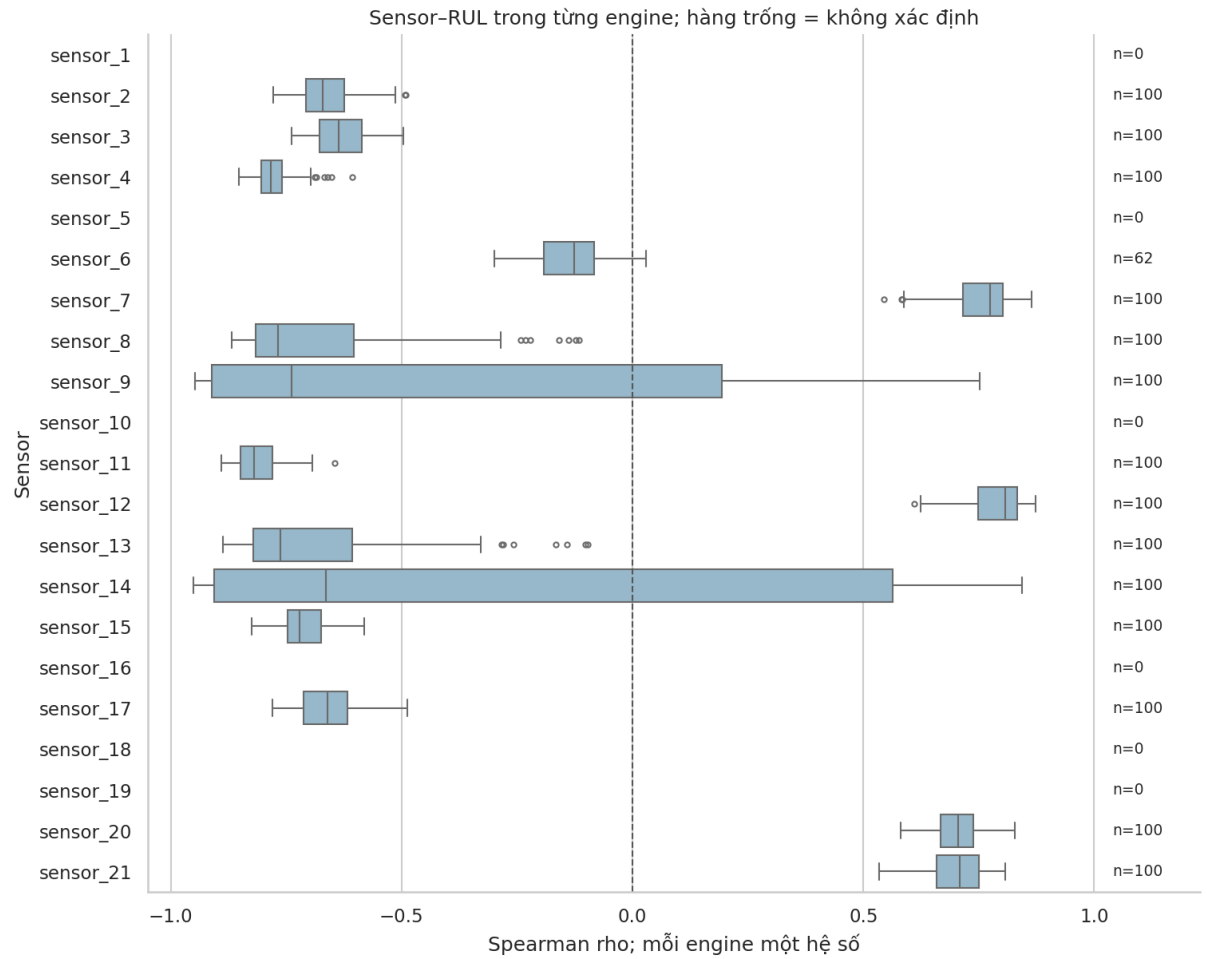
7.5 Sensor hằng và correlation theo engine

Metadata EDA ghi nhận `setting_3` và các sensor 1, 5, 10, 16, 18, 19 hằng trên train. Riêng sensor 6 hằng trong 38/100 động cơ; correlation sensor–RUL chỉ xác định trong 62 động cơ còn lại. Vì vậy, cần công bố số động cơ hợp lệ khi tóm tắt correlation.

Bảng 11: Một số dòng minh họa từ bảng correlation Spearman theo engine. Không dùng bảng này để xếp hạng hoặc chọn sensor cho mô hình.

Sensor	Động cơ xác định	Tương quan trung vị	Động cơ không xác định
1	0	Không xác định	100
6	62	−0,1264	38
11	100	−0,8190	0
12	100	0,8075	0

Sensor 11 có tương quan trung vị âm, trong khi sensor 12 có tương quan trung vị dương với RUL. RUL giảm theo cycle, nên dấu âm gợi ý sensor có xu hướng tăng khi tiến gần cuối vòng đời trong cách tổng hợp này. Đây là liên hệ mô tả trên train; chưa có kiểm định, chưa điều chỉnh multiple testing và chưa đo hiệu quả dự đoán ngoài mẫu.



Hình 4: Phân phối correlation Spearman sensor–RUL giữa các động cơ train. Sensor hằng được giữ là không xác định; mỗi hệ số hợp lệ tương ứng một động cơ.

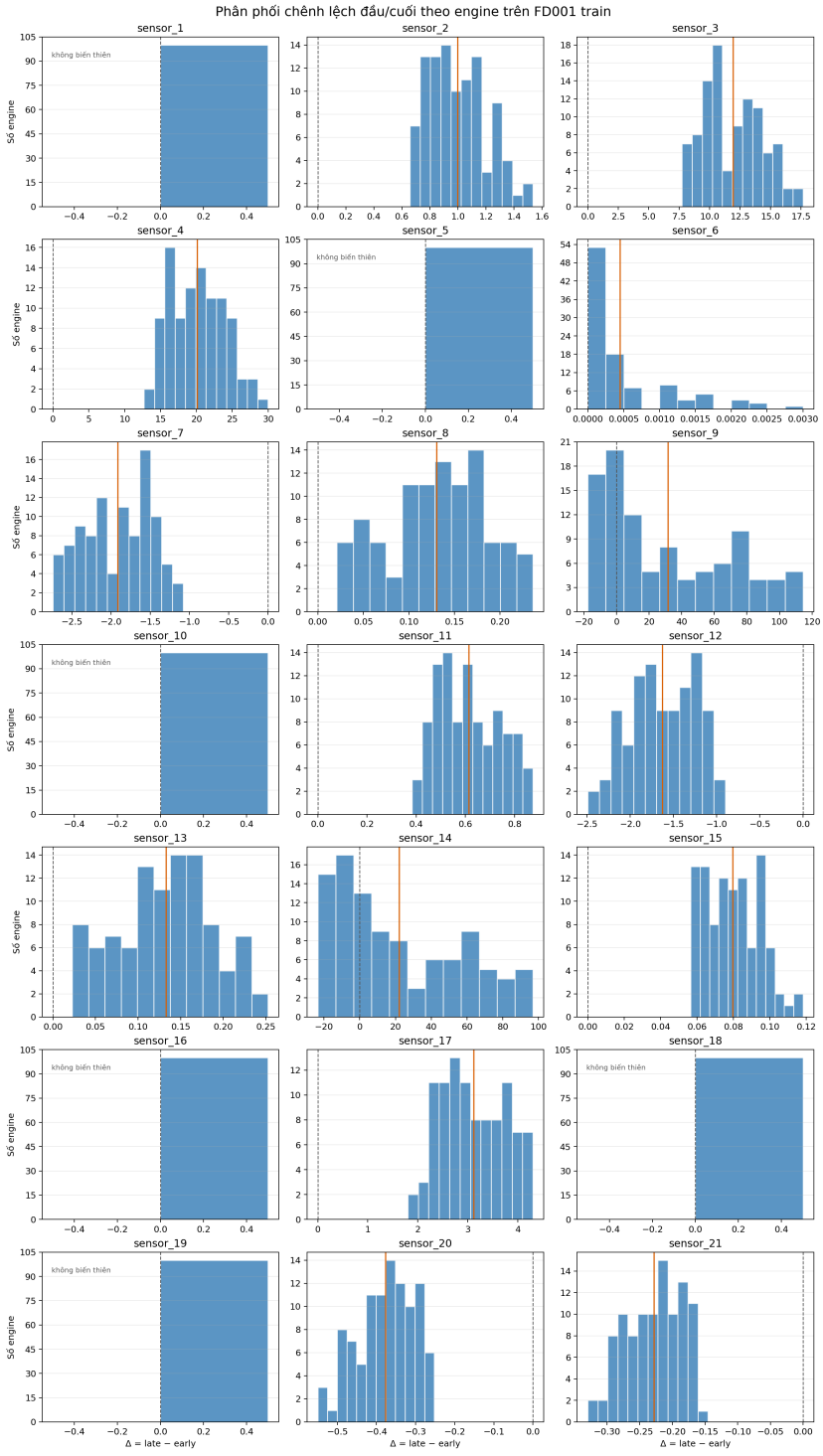
7.6 Suy luận đầu/cuối vòng đời trên train

Notebook `notebooks/02_fd001_inference.ipynb` đã tạo artifact M2.3a tại `outputs/statistics/`. Bảng cặp có 2.100 dòng, tương ứng 100 động cơ train nhân 21 sensor. Mỗi dòng dùng trung bình 30 cycles đầu, trung bình 30 cycles cuối và chênh lệch `late_mean - early_mean`. Bảng suy luận giữ đủ 21 sensor. Validation ghi `passed`, không đọc quan sát test hoặc nhãn test, đồng thời xác nhận SHA-256 của raw train không đổi.

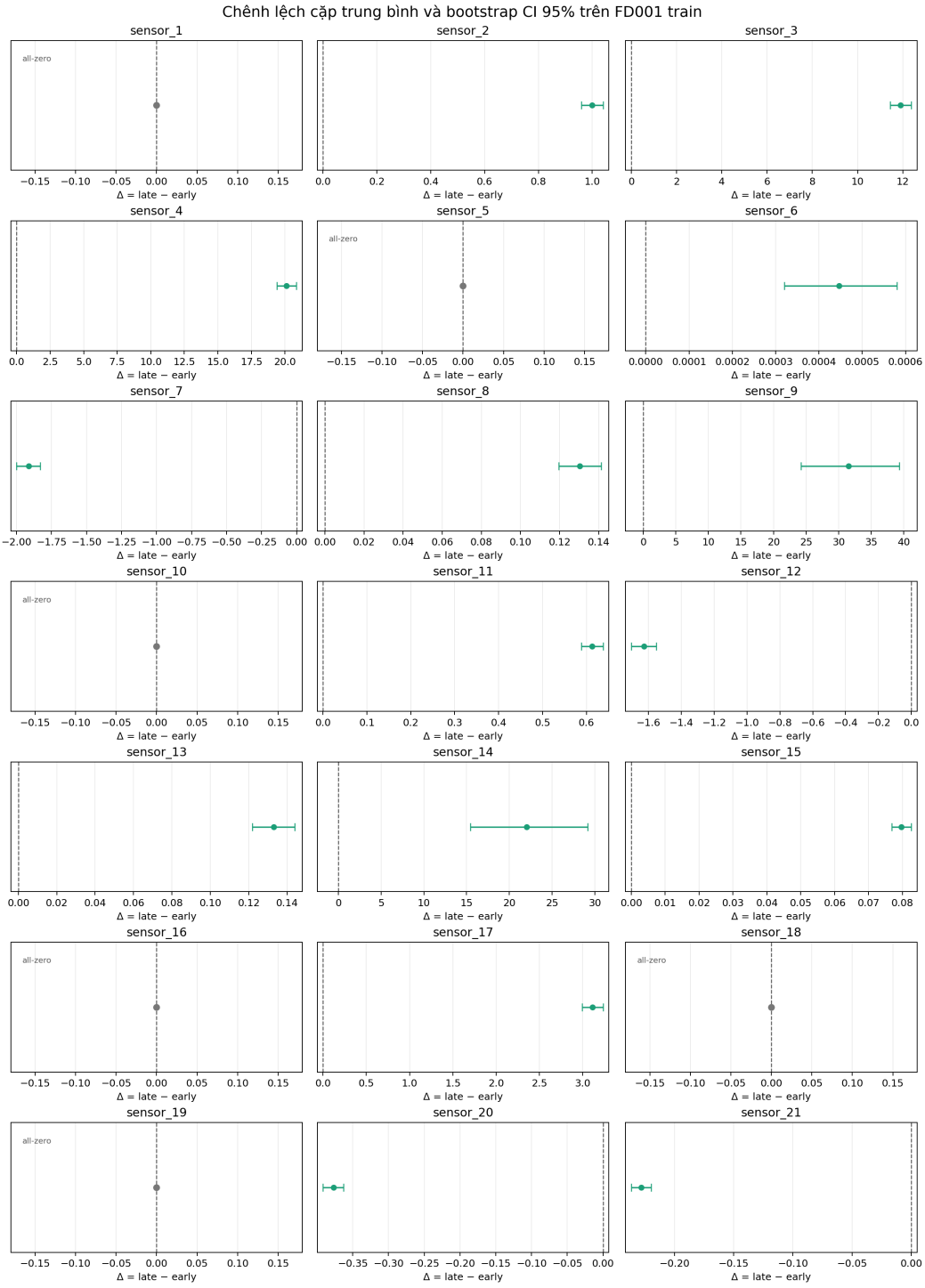
Trong 21 sensor, sáu sensor có toàn bộ chênh lệch bằng 0. Các sensor này là `sensor_1`, `sensor_5`, `sensor_10`, `sensor_16`, `sensor_18` và `sensor_19`. Chúng được giữ trong output với trạng thái `all_zero_difference`, khoảng tin cậy suy biến tại 0 và p-value không áp dụng. Mười lăm sensor còn lại kiểm định được; với cấu hình Wilcoxon permutation 100.000 resamples và Benjamini–Hochberg trên họ 21 sensor, các p-value đã hiệu chỉnh đều xấp xỉ 0,000028.

Bảng 12: Một số dòng từ `sensor_inference.csv`. Giá trị là chênh lệch cuối trừ đầu theo đơn vị gốc của từng sensor; không so sánh trực tiếp độ lớn giữa các sensor khác thang đo.

Sensor	Chênh lệch trung bình	Khoảng tin cậy bootstrap 95%	Dấu chính
`sensor_4`	20,128097	[19,402735; 20,865606]	Tăng
`sensor_7`	-1,913527	[-1,999307; -1,829684]	Giảm
`sensor_11`	0,613257	[0,589017; 0,637944]	Tăng
`sensor_12`	-1,627190	[-1,704288; -1,552119]	Giảm
`sensor_20`	-0,377007	[-0,391534; -0,363173]	Giảm



Hình 5: Phân phối `difference = late_mean - early_mean` của 100 engine cho từng sensor. Mỗi panel có thang x riêng vì các sensor dùng đơn vị và khoảng giá trị khác nhau; đường đứt là mốc không thay đổi, đường cam là mean paired difference.



Hình 6: Mean paired difference và bootstrap CI 95% theo từng sensor. Các panel giữ trục riêng để không tạo so sánh effect size giữa các sensor khác thang đo; điểm xám là sensor toàn bộ chênh lệch bằng 0.

Kết quả này cho thấy nhiều sensor có thay đổi có hệ thống giữa hai đoạn 30 cycles trên train FD001 theo protocol đã chốt. Tuy nhiên, đây không phải bằng chứng chọn đặc trưng cho mô hình trên toàn train. Việc chọn feature, scaler hoặc model vẫn phải nằm trong training folds để tránh data leakage.

Diễn giải hình. Sáu sensor all-zero nêu ở trên không có biến thiên quan sát được trong hai đoạn ghép cặp. Mười lăm sensor còn lại đủ điều kiện kiểm định; theo cấu hình Wilcoxon permutation và BH FDR đã chốt trước khi chạy, các sensor này đều bác bỏ giả thuyết không. Đây là bằng chứng thống kê trong protocol đầu/cuối 30 cycles, không phải bằng chứng rằng các sensor này có utility dự đoán RUL ngoài mẫu.

Độ lớn thay đổi cần được đọc cùng `mean_difference` và khoảng tin cậy theo đơn vị gốc của từng sensor. `sensor_4` có chênh lệch trung bình khoảng 20,13, trong khi `sensor_7` giảm khoảng 1,91; hai giá trị này không được xếp hạng trực tiếp vì thang đo khác nhau. `sensor_6` có 47 chênh lệch dương và 53 chênh lệch bằng 0 nhưng mean difference chỉ khoảng 0,00045, cho thấy p-value nhỏ không đồng nghĩa với thay đổi lớn về đơn vị đo. `sensor_9` có 71 chênh lệch dương và 29 chênh lệch âm, nên phân phối theo engine cần được xem cùng effect size và CI thay vì chỉ đọc mean.

7.7 Phạm vi bằng chứng EDA và thống kê

Các số liệu trên được đối chiếu với bảng CSV và metadata hiện có. Các phép tổng hợp được trích từ train cũng đã được kiểm tra lại trong lượt tạo báo cáo. Bản PDF tham chiếu trực tiếp hai hình EDA; không vẽ lại giá trị hoặc thay số để minh họa.

Biên bản `outputs/eda/validation.json` ghi nhận 69 tests đạt; notebook chạy từ kernel mới và được kiểm tra độc lập trong thư mục chỉ có train input. Runtime audit chặn việc đọc hai file test trong lần kiểm tra. Mười hai CSV và 11 PNG của hai lần chạy giống nhau theo byte trong cùng môi trường.

Đây là bằng chứng đã lưu của mốc EDA, không phải kết quả chạy lại notebook trong lượt viết báo cáo. Mốc EDA mô tả không bao gồm feature selection hoặc kết quả mô hình. Với mốc thống kê, `outputs/statistics/validation.json` ghi nhận focused statistics tests đạt 26 tests, full suite đạt 95 tests, notebook 02 chạy hai lần từ fresh kernel, hai CSV và hai PNG giống nhau theo SHA-256 trong cùng môi trường. Runtime audit `STATISTICS_TRAIN_ONLY` cho phép đọc `train_FD001.txt` và chặn probe đọc `test_FD001.txt` cùng `RUL_FD001.txt`. Đây là bằng chứng validation cục bộ, không phải filesystem sandbox, CI enforcement hoặc branch protection. Tính đồng nhất từng byte của PNG không được bảo đảm trên môi trường có font hoặc rendering khác.

8 Khó khăn, bài học và giới hạn hiện tại

Khó khăn đầu tiên là phân biệt dữ liệu đủ để tính nhãn với dữ liệu đủ để dự đoán trong thực tế. Train cho phép biết T_i sau khi chuỗi kết thúc, còn tại thời điểm dự đoán t thì T_i chưa biết. Bài học thiết kế là tách việc tạo nhãn khỏi feature và không dùng tuổi thọ đầy đủ để tạo biến đầu vào dự đoán.

Khó khăn thứ hai là xác định đơn vị phân tích. Một engine sống lâu tạo nhiều dòng hơn và các quan sát liền nhau có dependence theo thời gian. Việc tăng số dòng không tương đương tăng số mẫu độc lập. EDA vì vậy ghi rõ mức observation hoặc engine; validation và bootstrap dự kiến giữ đơn vị engine.

Khó khăn thứ ba là giới hạn bằng chứng của dữ liệu mô phỏng và quy mô nhỏ. Benchmark PySpark có thể chịu overhead lớn; chưa thể giả định phân tán sẽ nhanh hơn. Kết quả âm hoặc tốc độ thấp hơn Pandas vẫn cần được trình bày nếu phép đo hợp lệ. Báo cáo chỉ phân tích khả năng mở rộng sau khi có parity và thời gian đo thực tế.

Môi trường kiểm thử trước đó ghi nhận lỗi quyền truy cập thư mục Temp trên Windows. Cách xử lý đã được tài liệu hóa là dùng thư mục tạm riêng cho pytest; đây là vấn đề môi trường, không phải lỗi chất lượng FD001. Trong lượt khởi tạo báo cáo, TeX chưa có trên PATH nên dùng Tectonic portable để biên dịch; hướng dẫn tái lập được lưu cùng nguồn báo cáo.

Hiện đã có khoảng tin cậy bootstrap cho thay đổi sensor đầu/cuối trên train, nhưng chưa có kết quả huấn luyện, metric test, khoảng tin cậy cho metric mô hình, ARIMA diagnostics hoặc dashboard. Do đó, chưa thể kết luận mô hình nào phù hợp hơn, cảnh báo sớm chính xác đến đâu hoặc lịch bảo trì nào có lợi. Phần kết luận cuối sẽ được viết sau khi hoàn tất đánh giá theo protocol đã lưu.

9 Khai báo sử dụng AI/LLM

AI/LLM được sử dụng để hỗ trợ xây dựng cấu trúc bản báo cáo, diễn đạt nội dung, tổ chức nhật ký tiến độ, đối chiếu artifact trong repository và hỗ trợ kiểm tra biên dịch. Nội dung hỗ trợ này cần được kiểm tra với dữ liệu, mã nguồn và tài liệu gốc. Việc tạo văn bản bằng AI không được dùng làm bằng chứng về hiệu quả mô hình; các kết quả thực nghiệm chỉ được ghi nhận khi có artifact và quy trình kiểm chứng tương ứng.

Tài liệu tham khảo

[1] Apache Software Foundation. Spark SQL and DataFrames. Apache Spark documentation. Accessed 2026-09-22; implementation version not selected. URL: https://spark.apache.org/docs/latest/sql-programming-guide.html.

[2] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi:10.1111/j.2517-6161.1995.tb02031.x.

[3] B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi:10.1214/aos/1176344552.

[4] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3 edition, 2021. Online edition, sections 5.4, 9.1, 9.5 and 9.7; accessed 2026-09-22. URL: https://otexts.com/fpp3/.

[5] Xiang Li, Qian Ding, and Jian-Qiao Sun. Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety*, 172:1–11, 2018. doi:10.1016/j.ress.2017.11.021.

[6] NASA. CMAPSS jet engine simulated data. NASA Open Data Portal. Accessed 2026-09-20; project uses FD001. URL: https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data.

[7] Emmanuel Ramasso and Abhinav Saxena. Review and analysis of algorithmic approaches developed for prognostics on CMAPSS dataset. In *Annual Conference of the PHM Society*, volume 6, 2014. Issue 1. URL: https://papers.phmsociety.org/index.php/phmconf/article/view/2512, doi:10.36001/phmconf.2014.v6i1.2512.

[8] Abhinav Saxena, Jose Celaya, Bhaskar Saha, Sankalita Saha, and Kai Goebel. Metrics for offline evaluation of prognostic performance. *International Journal of Prognostics and Health Management*, 1(1), 2010. URL: https://papers.phmsociety.org/index.php/ijphm/article/view/1336, doi:10.36001/ijphm.2010.v1i1.1336.

[9] Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management*, pages 1–9. IEEE, 2008. doi:10.1109/PHM.2008.4711414.

[10] scikit-learn developers. Cross-validation iterators for grouped data. User Guide. Accessed 2026-09-22; implementation version not selected. URL: https://scikit-learn.org/stable/modules/cross_validation.html#cross-validation-iterators-for-grouped-data.

[11] scikit-learn developers. Decomposing signals in components: Principal Component Analysis. User Guide. Accessed 2026-09-22; implementation version not selected. URL: https://scikit-learn.org/stable/modules/decomposition.html#pca.

[12] scikit-learn developers. Histogram-Based Gradient Boosting. User Guide. Accessed 2026-09-22; implementation version not selected. URL: https://scikit-learn.org/stable/modules/ensemble.html#histogram-based-gradient-boosting.

[13] scikit-learn developers. Linear models: Ridge regression and classification. User Guide. Accessed 2026-09-22; implementation version not selected. URL: https://scikit-learn.org/stable/modules/linear_model.html#ridge-regression-and-classification.

[14] SciPy developers. scipy.stats.bootstrap. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.bootstrap.html.

[15] SciPy developers. scipy.stats.false_discovery_control. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.false_discovery_control.html.

[16] SciPy developers. scipy.stats.wilcoxon. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.wilcoxon.html.

[17] statsmodels developers. statsmodels.tsa.stattools.adfuller. Online documentation. Accessed 2026-09-22; implementation version not selected. URL: https://www.statsmodels.org/stable/generated/statsmodels.tsa.stattools.adfuller.html.

[18] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi:10.2307/3001968.

[19] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 15–28. USENIX Association, 2012. URL: https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia.

A Bảng đối chiếu nội dung và bằng chứng

Các đường dẫn sau được tính từ thư mục gốc repository. Hai tài liệu yêu cầu môn học nằm trong thư mục Resources bên ngoài repository và chưa được sao chép vào gói báo cáo.

Nội dung	Nguồn kiểm tra
Quy mô, schema và data dictionary	`outputs/validation/fd001.json`; `data/CONTRACTS.md`; `outputs/eda/tables/schema.csv`; `outputs/eda/tables/measurement_summary.csv`.
Số nhãn test	`outputs/validation/fd001-integrity.json`; không sử dụng giá trị nhãn để phân tích trong bản này.
Nguồn và phạm vi FD001	`data/raw/DATA_SOURCE.md`; `data/raw/readme.txt`; `references/bibliography.bib`.
Cơ sở lý thuyết và mức đọc nguồn	`report/sections/theory.tex`; `references/README.md`; `references/reading-guide.md`. Các nguồn bài gốc chưa đọc toàn văn được ghi rõ.
Protocol mốc 2	`report/sections/statistical-protocol.tex`; mean 30 cycles đầu/cuối và cấu hình M2.0 đã chốt ngày 22/09/2026. Sơ đồ có nguồn tại `report/figures/statistical-protocol.tex`.
Tạo cặp, suy luận và tái lập mốc 2	`src/cmapss/statistics/`; `tests/test_statistics_pairs.py`, `tests/test_statistics_inference.py`, `tests/test_statistics_artifacts.py` và `tests/test_statistics_reproducibility.py`; contract tại `data/CONTRACTS.md`. M2.3a đã lưu `outputs/statistics/tables/engine_sensor_pairs.csv`, `outputs/statistics/tables/sensor_inference.csv`, `outputs/statistics/run_info.json` và `outputs/statistics/validation.json`. M2.3b đã lưu `outputs/statistics/figures/difference_distributions.png` và `outputs/statistics/figures/effect_size_ci.png`. M2.4 ghi biên bản trong `outputs/statistics/validation.json`: 2.100 cặp, 21 dòng suy luận, focused tests 26, full suite 95, notebook chạy hai lần từ fresh kernel, hai CSV và hai PNG giống nhau theo SHA-256, runtime audit chặn `data/raw/test_FD001.txt` và `data/raw/RUL_FD001.txt`.

Nội dung	Nguồn kiểm tra
ETL Silver/Gold và benchmark M3.1	`src/cmapss/etl/`; `tests/test_etl.py`; contract ETL tại `data/CONTRACTS.md`. `outputs/etl/run_info.json` ghi 33.727 observations, 20.631 train targets, 200 engine rows và checksum raw. `outputs/etl/benchmark.json` ghi benchmark Pandas 1×/10×/100×, hai lần lặp cold/warm. `outputs/etl/parity.json` ghi Pandas/PySpark parity đạt cho Silver observations, Silver train targets và Gold engine summary, với `max_abs_diff=0.0`. Parquet và SQLite được đọc lại trong test.
Rolling features M3.2	`src/cmapss/features/`; `tests/test_features.py`; contract rolling tại `data/CONTRACTS.md`. `outputs/features/run_info.json` ghi window 30 cycles, 27.927 rows, 17.731 train windows, 10.196 test windows, 126 feature columns, sample `std` với `ddof=1`, `joins_test_labels=false` và `modeling.status=not_started`. `outputs/features/gold/rolling_features.parquet` được đọc lại trong test.
Tuổi thọ động cơ	`outputs/eda/tables/engine_lifetimes.csv`; `outputs/eda/tables/lifetime_summary.csv`.
Sensor hằng	`outputs/eda/run_info.json`; `outputs/eda/tables/constant_engine_counts.csv`.
Correlation theo engine	`outputs/eda/tables/sensor_rul_by_engine.csv`; `outputs/eda/tables/sensor_rul_summary.csv`.
Nguồn EDA	`notebooks/01_fd001_eda.ipynb`; `src/cmapss/eda.py`.
Nghiệm thu EDA mô tả	`outputs/eda/validation.json`; 69 tests, kernel mới, 12 CSV và 11 PNG tái lập trong cùng môi trường.
Kiểm tra số liệu được trích	`report/evidence/report-checks.txt`; kiểm tra chỉ đọc, không chạy huấn luyện hoặc sửa artifact EDA.
Nguồn từng phần báo cáo	`report/sections/`; hướng dẫn cập nhật và build tại `report/README.md`.

Ví dụ đối chiếu: giá trị mean tuổi thọ 206,31 trong Bảng 10 lấy từ dòng `mean` của `lifetime_summary.csv`; file chi tiết có một dòng cho mỗi engine. Không lấy trung bình trực tiếp cột cycle trên toàn bộ observations vì đó là một đại lượng khác.

B Thông tin cần bổ sung trước khi nộp

Mẫu bìa chính thức, tên giảng viên, mã lớp và thông tin đăng ký thực hiện cá nhân chưa được xác nhận trong phạm vi tài liệu hiện có. Các mục này cần được đối chiếu trước khi nộp; không giả định đã được phê duyệt. Chưa có thời hạn theo ngày lịch để chuyển thành kế hoạch tuần cụ thể. Các mốc tại Bảng 3 vì vậy chỉ ghi thứ tự và điều kiện hoàn thành.